



BCSE498J- Project - II

AUTOMATED CICD FOR EMOTION AWARE NLP APPLICATION

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

22BCE0927 AYUSH GUPTA

Under the Supervision of

Prof. Meenakshi S P

Assistant Professor Sr. Grade 2

School of Computer Science and Engineering



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

April, 2026

DECLARATION

I hereby declare that the project entitled “AUTOMATED CICD FOR EMOTION AWARE NLP APPLICATION” submitted by me, for the award of the degree of Bachelor of Technology in Computer Science and Engineering to VIT is a record of bonafide work carried out by me under the supervision of Prof. Meenakshi S P, School of Computer Science and Engineering.

I further declare that the work reported in this project report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

AYUSH GUPTA (22BCE0927)

Date :

CERTIFICATE

This is to certify that the project entitled “AUTOMATED CICD FOR EMOTION AWARE NLP APPLICATION” submitted by AYUSH GUPTA (22BCE0927), School of Computer Science and Engineering, VIT, for the award of the degree of Bachelor of Technology in Computer Science and Engineering , is a record of bonafide work carried out by the student under my supervision during Winter Semester 2025-2026, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore

Signature of the Guide

Date :

Internal Examiner

External Examiner

Head of the Department

ACKNOWLEDGEMENT

I AYUSH GUPTA (22BCE0927) am deeply grateful to the management of Vellore Institute of Technology (VIT) for providing me with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in my academic journey. The support and infrastructure provided by VIT have enabled me to explore and develop my ideas to their fullest potential.

My sincere thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering, for constant support and encouragement. The Dean's leadership and vision have greatly inspired me to strive for excellence. The dedication to academic excellence and innovation has been a constant source of motivation.

I express my profound appreciation to Dr. Boominathan P, the Head of the Department of Software Systems for insightful guidance and continuous support. The expertise and advice have been crucial in shaping the direction of my project. The commitment to fostering a collaborative and supportive atmosphere has greatly enhanced my learning experience. The constructive feedback and encouragement have been invaluable in overcoming challenges and achieving my project goals.

I am immensely thankful to my project supervisor, Prof. Meenakshi S P, School of Computer Science and Engineering, for dedicated mentorship and invaluable feedback. The patience, knowledge, and encouragement have been pivotal in the successful completion of this project. The willingness to share expertise and provide thoughtful guidance has been instrumental in refining my ideas and methodologies. This support has not only contributed to the success of this project but has also enriched my overall academic experience.

I extend my heartfelt thanks to all for the guidance and support received throughout this endeavor.

TABLE OF CONTENTS

Executive Summary	iii
List of Tables	iv
List of Figures	v
List of Abbreviations	vi
List of Symbols	vii
1 INTRODUCTION	1
1.1 Background	1
1.1.1 Existing Software Delivery Practices	2
1.1.2 Need for Automated CI/CD Pipelines	2
1.2 Motivation	3
1.3 Scope of the project	4
2 PROJECT DESCRIPTION AND GOALS	6
2.1 Literature Review	6
2.2 Research Gap	8
2.3 Objectives	9
2.4 Problem Statement	10
2.5 Project Plan	11
3 TECHNICAL SPECIFICATION	14
3.1 Technical Specifications	14
3.1.1 Functional Requirements	14
3.1.2 Non-Functional Requirements	15
3.1.3 System Requirements	16
3.2 Feasibility Study	18
3.3 System Specification	19
3.4 Mathematical Model	20
3.4.1 Algorithm	21
4 SYSTEM DESIGN	22
4.1 System Architecture	22
4.1.1 Continuous Integration with Jenkins Multibranch Pipeline	24

4.1.2	GitOps Deployment with Argo CD	25
4.2	Design	26
4.2.1	Data Flow Diagram	26
4.2.2	Use Case Diagram	28
4.2.3	Class Diagram	28
4.2.4	Sequence Diagram	29
5	METHODOLOGY AND TESTING	35
5.1	Overview	35
5.1.1	Methodology	35
5.2	Module Description	38
5.3	Testing Approach	39
6	PROJECT IMPLEMENTATION	42
6.1	System Configuration	42
6.2	Feature Demonstration	43
6.3	Testing Scenarios	44
6.4	Expected Outcomes	45
6.5	Application Containerisation	46
6.6	Jenkins Pipeline Implementation	47
6.7	Kubernetes Manifest Implementation	50
6.8	Argo CD Application Implementation	51
6.9	Observability Implementation	52
6.10	Implementation Summary	53
7	RESULTS AND DISCUSSION	55
7.1	Experimental Results	55
7.2	Latency Analysis	56
7.3	Resource Utilization Analysis	57
7.4	Deployment Reliability Analysis	59
7.5	Security Posture Analysis	59
7.6	Cost Analysis	60
7.7	Discussion	61
8	CONCLUSION AND FURTHER ENHANCEMENTS	62
8.1	Conclusion	62
8.2	Further Enhancements	63

EXECUTIVE SUMMARY

The delivery process for software has been revolutionized. What once took days, even weeks, worth of iterative testing and development work between software engineers and their operations colleagues can now be accomplished multiple times daily via automation. CI/CD, Kubernetes, and GitOps facilitated the evolution — yet constructing a pipeline that can function effectively in reality is more difficult than any tutorial demonstrates.

This project delivers an end-to-end CI/CD pipeline for a containerized machine learning application. The application itself is a sentiment analysis service based on Flask, employing XGBoost, scikit-learn, and NLTK to classify texts into positive and negative sentiments. Once the developer commits to the main branch, the automated pipeline kicks into gear, constructing a Docker image, uploading it to Docker Hub, syncing the Kubernetes configuration to Git, and finally, deploying it onto an AWS EKS cluster.

The system itself is divided into two parts. Jenkins takes care of CI and lives in an EC2 instance, while Argo CD takes care of CD and runs inside the cluster, monitoring Git for changes. Jenkins doesn't interact with the cluster at all, meaning it never gets cluster credentials — a security feature on purpose. Git history becomes the deployment history.

In addition to the architecture, Docker image builds use a multi-stage approach, are run as a non-root user, and pre-cache their NLTK corpora. Kubernetes deploys a service with resource limits, health checks, and two instances for zero downtime during rolling updates. Prometheus, Grafana, and Alertmanager are responsible for observability.

We deployed the system on AWS several times in our actual AWS account and experienced issues that none of the tutorials mentioned before, such as GPG key expiration, Java version conflicts, an undocumented deprecation of a package repository, and orphaned load balancer billing us non-stop till we discovered it. Being able to address these challenges honestly is crucial to this report.

List of Tables

1.1	Comparison of traditional and modern software delivery practices . . .	2
2.1	Comparison of existing approaches to continuous deployment	7
3.1	System requirement specification	17
3.2	Cost profile of the deployed pipeline	19
3.3	System modules and functionalities	20
4.1	Architecture layer description	24
5.1	Functional testing results	41
6.1	Implementation module summary	54
7.1	Cycle time comparison	57
7.2	Resource utilisation analysis	58
8.1	Possible future enhancements	65

List of Figures

1.1	General architecture of a modern CI/CD pipeline	3
1.2	Motivation for an automated CI/CD pipeline	4
2.1	Comparison of existing CI/CD approaches	7
2.2	Identified research gap	9
2.3	Project execution plan	13
3.1	AWS infrastructure setup showing EC2 and EKS resources	17
4.1	Overall system architecture	23
4.2	Detailed module interaction	24
4.3	Level 0 – Context Diagram	26
4.4	Level 1 – Detailed Data Flow: Core Processes	31
4.5	Level 2 – Detailed Processes within the Pipeline	32
4.6	Use Case Diagram	33
4.7	Class Diagram	33
4.8	Sequence Diagram	34
5.1	Methodology workflow of the proposed pipeline	38
6.1	Docker image build output	47
6.2	Jenkins multibranch pipeline view	49
6.3	Argo CD application in synced and healthy state	53
6.4	Grafana dashboard showing cluster metrics	54
7.1	Cycle time comparison graph	57
7.2	Resource utilisation observed in Grafana	58

LIST OF ABBREVIATIONS

API	Application Programming Interface
AWS	Amazon Web Services
CD	Continuous Deployment / Continuous Delivery
CI	Continuous Integration
CLI	Command Line Interface
CRD	Custom Resource Definition
EBS	Elastic Block Store
EC2	Elastic Compute Cloud
EKS	Elastic Kubernetes Service
ELB	Elastic Load Balancer
GPG	GNU Privacy Guard
HPA	Horizontal Pod Autoscaler
IAM	Identity and Access Management
JSON	JavaScript Object Notation
ML	Machine Learning
NLTK	Natural Language Toolkit
PAT	Personal Access Token
PR	Pull Request
REST	Representational State Transfer
SCM	Source Control Management
SPOF	Single Point of Failure
VCS	Version Control System
WSGI	Web Server Gateway Interface
YAML	YAML Ain't Markup Language

SYMBOLS AND NOTATIONS

μ	Memory utilisation of a pod
ρ	CPU utilisation ratio of a pod against its requested allocation
N	Build number assigned by Jenkins to a pipeline execution
R	Number of running replicas in a deployment
T_b	Time taken for image build and push stage
T_d	Time taken for deployment reconciliation by Argo CD

CHAPTER 1

INTRODUCTION

1.1 Background

It is not the same process of software delivery as it was fifteen years ago. The previous practice, where there were different teams coding software, then packaging it, and then deploying it to servers manually, has been slowly replaced by a continuous and automated flow of software delivery, which is close to self-service. There were several factors behind this change. First, cloud computing became inexpensive and widespread. Second, containerization became the new norm for packaging software. Third, Kubernetes came into prominence as a platform to orchestrate containers. Fourth, the DevOps paradigm helped align software engineers and operations. (Humble and Farley, 2010; Kim et al., 2016).

The expectations shifted along with the tooling. Companies deploy their software several times a day, and each deployment needs to be reliable, reproducible, and reversible. This speed is no longer negotiable — it’s simply the reality of modern development teams — and it explains why CI/CD pipelines went from being a “nice to have” to essential infrastructure. A CI/CD pipeline, most fundamentally, is an automated flow that code follows from the developer’s editor all the way to production, where every step is specified via configuration stored in version control with the code (Fowler, 2006; Bass et al., 2015).

The abstract concept is clear enough. However, implementing a pipeline resembling an actual production environment continues to be more difficult than expected. A lot of guides either talked only about the ideal situation or used outdated tools. Even more surprisingly, some of the resources were completely oblivious to any security concerns. And then the issue becomes even more complicated when it comes to building a deployment pipeline for a ML application because there are various artifacts to consider (models, native dependencies, NLTK corpora, etc.)

This is precisely what our project aimed to resolve. Specifically, we developed and tested an CI/CD pipeline that includes all necessary steps for deploying a Flask web application with an XGBoost classifier trained on a dataset of movie reviews. The components include Jenkins for CI, Docker for building the container image, EKS for cluster management, and Argo CD for GitOps deployment.

1.1.1 Existing Software Delivery Practices

Before the adoption of CI/CD, applications were released using what was known as a release train. This entailed the accumulation of features over weeks or months before being launched together in one grand push. Such a deployment was quite fragile for various reasons, which now appear quite obvious. Deployment itself was hardly ever practiced. So much had been changed that debugging which change broke things would be like detective work. Meanwhile, reverting the change would be a painstaking process carried out by the unfortunate soul chosen to do so, typically around 2am. The concept of Continuous Integration was designed to solve this issue on the merging end of the equation. As per the concept, developers are supposed to integrate their code into a common trunk regularly. Automated builds and testing would run upon each merge to identify issues early enough when they are small. Continuous Deployment does the same thing but takes it a step further beyond mere building and into production. If done right, the actual merging of the code becomes tantamount to a release

Table 1.1: Comparison of traditional and modern software delivery practices

Parameter	Traditional Release	Modern CI/CD
Release frequency	Weekly to quarterly	Multiple times per day
Manual steps	Many	Few or none
Rollback time	Hours	Minutes
Auditability	Limited	Complete via Git history
Risk per release	High	Low

This change is summarized by Table 1.1. The key is not simply that CI/CD reduces time to release; it allows for releases to be faster, more compact, safer, and more manageable in their complexity.

1.1.2 Need for Automated CI/CD Pipelines

There are three main reasons for doing pipeline automation. The first reason is reliability. There are many inconsistencies in manual processes, despite documentation of all procedures. With an automated pipeline, the exact same steps are executed each time, based on the same input files and credentials used each time. Predictability, in turn, translates into reliability.

The second reason is speed. The time interval between "I made a fix" and "end-users have the fix" is significant. Not only does it impact the ability to patch security vulnerabilities quickly, but it also affects incident resolution and iteration speed. If teams reduce that interval down to minutes, their capacity becomes different entirely.

Lastly, auditability might be considered the least discussed but the most important

one for many enterprises. Every deployment made through automation comes down to the exact Git commit, exact pipeline execution, and exact reconciliation performed by that pipeline. Therefore, the full history of deployments, who did them, where, when and based on which code versions sits right there in Git logs.

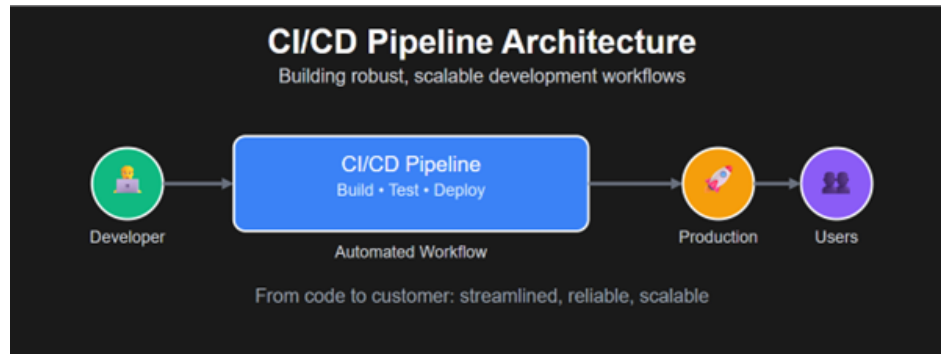


Figure 1.1: General architecture of a modern CI/CD pipeline

1.2 Motivation

This project arose out of a combination of academic and pragmatic motivations. From an academic perspective, CI/CD systems and cloud-native application deployments are subjects that get referenced constantly in courses, industry blogs, and even job postings, yet receive almost no practical consideration whatsoever. It's one thing to study Kubernetes. It's quite another to build a cluster, watch it fall apart due to faulty probe configuration, debug it, and have your AWS costs silently accumulate as you do so.

From a pragmatic point of view, there's the issue of finding employment. The software engineering, site reliability engineer, and DevOps positions of today all assume familiarity with these technologies. One who has successfully deployed a build pipeline, seen it go haywire, debugged the issues, and possibly created an orphaned load balancer will be ahead of his counterparts, who simply know how to run the command lines.

There was also a third point of consideration, which would be more difficult to summarize. The vast majority of publicly accessible information about CI/CD consists of a vendor's demo that glosses over the complexities, a blog post published on Medium that has long since fallen into neglect, or an architectural diagram that is so theoretical it obfuscates any actual practical difficulty involved. The process of actually building the system introduces a continuous stream of little issues that cannot be found anywhere else. Outdated GPG signature keys. Repositories for tools that have silently migrated to another branch. Cloud services that are still being billed but have long been forgotten about.

As depicted by Figure 1.2, delays, inconsistencies, and other issues arise from the

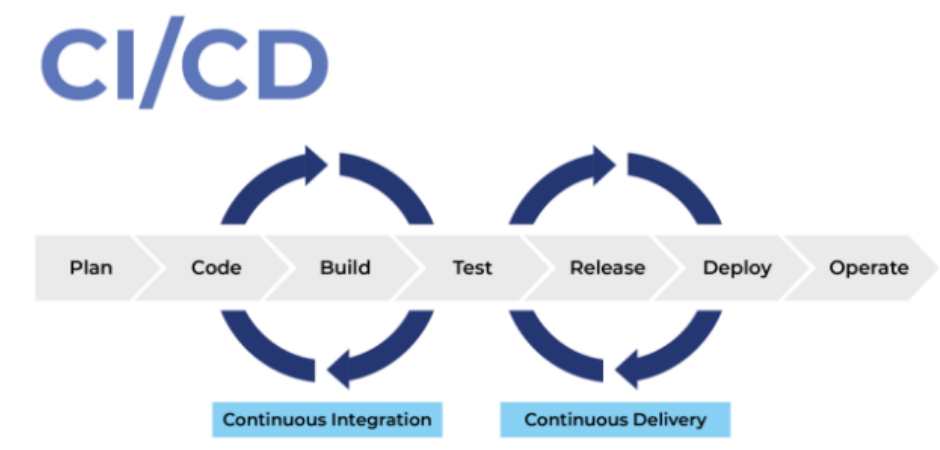


Figure 1.2: Motivation for an automated CI/CD pipeline

manual process of deployment, which are not even required to begin with. However, an appropriately configured pipeline eliminates many of these issues. The cost incurred for doing this is time investment for configuring tools, and a few instances of “why isn’t this working?”.

1.3 Scope of the project

The objective is clear-cut: develop a complete CI/CD pipeline that deploys an ML application as a containerized application through commonly used open source cloud native technologies. The pipeline will receive an input in the form of developer commit on the `main` branch, and from that point onwards, it will automatically perform a series of tasks to build a Docker image, push it to a public registry, update the Kubernetes deployment manifest, and deploy the newly developed version onto a Kubernetes cluster running on Amazon EC2.

The major components and scope of this project include:

1. **Containerising a Machine Learning Application:** The app is a Flask web service doing sentiment analysis using an XGBoost classifier, with text preprocessing handled by NLTK. We packaged it using a multi-stage Docker build that produces a small, hardened runtime image (Merkel, 2014).
2. **Continuous Integration Pipeline:** We deployed the Jenkins multibranch pipeline, monitoring the changes in the GitHub repo, triggering automatically on main branch commit, building the Docker image using the build number as tag, uploading the image to the Docker hub and updating the Kubernetes deployment file in the same repo.

3. **Continuous Deployment Using GitOps Approach:** Argo CD is running within the cluster and watching the Git repo for any changes in the deployment file. As soon as it detects a change in the deployment file, it pulls the new deployment file and applies it to the cluster, triggering a rolling update of the pods. By doing so, we manage to keep our CI system decoupled from the production cluster (Weaveworks, 2017).
4. **Infrastructure:** Our infrastructure uses AWS as a cloud provider with Jenkins master node hosted on EC2 and Kubernetes cluster created with EKS. EKS cluster was set up with the help of `eksctl` and contains one managed node group of two worker nodes.
5. **Observability:** Monitoring setup with tools such as Prometheus, Grafana, and Alertmanager is implemented using kube-prometheus-stack Helm chart. Prometheus collects metrics from the whole cluster and the application itself, while Grafana visualizes all this data through dashboards provided by the Helm chart (Turnbull, 2018).
6. **Production-Grade Practices:** We attempted to implement several practices that would be common in production but would not appear in tutorials. Those practices include not running containers as root user, specifying resource requests/limits in the deployment, setting up liveness/readiness probes, adding build number as image tag to simplify rollbacks, and ensuring Jenkins never has cluster credentials.
7. **Operational Validation:** The entire pipeline was executed from start to finish over multiple deployment phases to ensure that the commits indeed reach the cluster, rolling updates do not disrupt operations, and the pipeline can be shut down properly to avoid unexpected charges for any resources that have been overlooked.

The purpose of this project is to demonstrate how an end-to-end software deployment pipeline can be developed by one or two developers, using off-the-shelf technologies and modest cloud infrastructure costs — and in the process, creating something useful and learning something about operations that cannot necessarily be learned from a book.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

The literature dealing with continuous integration and deployment dates back to before the start of the millennium and its evolution is linked with the evolution of the tools. Initially, the focus of the research in this domain was on frequent integration as a solution to problems related to integration in large software development teams. In his now-classic paper on continuous integration, Fowler states that continuous integration with the practice of integration occurring multiple times a day and validated with automated builds significantly reduces the risks associated with the merge of long-lived branches (Fowler, 2006).

Continuous delivery and deployment as improvements upon continuous integration were extensively described by Humble and Farley, who positioned the deployment pipeline at the very heart of software delivery practices (Humble and Farley, 2010). In particular, they defined the deployment pipeline as the chain of steps taken by the software change and associated the level of progress with the increasing certainty in the appropriateness of the software for production use. At the same time, they distinguished two types of pipelines: the *continuous delivery*, which involved manual triggering of releases, and the *continuous deployment*, where automation extended to every part of the process.

After that came containers. The most prominent representative of this new class of software technologies is the technology of Docker. Merkel introduces the concept, which consists in bundling up the application along with all the dependencies into an easy-to-use package that will work exactly the same way both in a developer's environment, CI, and in production (Merkel, 2014). Containers solved one of the oldest problems in software deployments: the difference in behavior between a local build and a production one. Surely, operating vast amounts of containers across multiple machines poses a challenge that isn't solved by simply having runtimes to operate on. Kubernetes, developed in light of the company's use of Borg, became the most popular orchestrator. It provided primitives for declarative deployments, automatic service discovery, scalable deployment and self-healing systems (Burns et al., 2016). The model of the Kubernetes system, where the state is described in YAML files, and the platform

makes sure the actual state corresponds to the desired one, happened to be particularly well-suited for version control. And this is the starting point for all further developments.

Developed by Weaveworks, the GitOps paradigm relies explicitly on the idea of declarative management. In GitOps, Git itself becomes a single source of truth concerning desired states of any managed system (Weaveworks, 2017). According to GitOps, the CI system does not push changes into the cluster. On the contrary, an in-cluster controller polls changes from Git and enforces them. This change in the direction of operation brings a lot of additional benefits; first, better security due to the elimination of cluster credential storage from the CI process; second, improved auditability because the history of commits serves as a record of deployments history,

Later research moved in the direction of utilizing CI/CD for managing ML operations, whereby additional considerations such as data management, artifact management, and reproducibility become involved, which adds additional layers to the existing CI/CD pipeline (Sculley et al., 2015; Treveil et al., 2020). Our proposed solution is not intended to tackle the MLOps challenge, as a whole; however, it tackles the issue of shipping an ML service with proper CI/CD practices in mind.

Table 2.1: Comparison of existing approaches to continuous deployment

Approach	Security	Auditability	Self-Healing
Manual deployment	Variable	Low	None
Push-based CD (Jenkins to cluster)	Medium	Medium	None
Pull-based CD (GitOps)	High	High	Yes
Proposed pipeline (Jenkins + Argo CD)	High	High	Yes

As shown in Table 2.1, our pipeline takes the best of both worlds by incorporating the advantages of an established CI tool along with the practicality of GitOps. What you get is a system that seems natural to anyone familiar with the conventional CI/CD process while also adhering to the way cloud-based deployments take place in reality.

Figure 2.1 shows the compromise between the different approaches currently available. Manual approach allows complete control over the process, but there is no automation at all. In push-based CD, automation is gained at the expense of tying up the CI component with the cluster and creating the credential issue discussed above. With GitOps, automation is achieved with separation between the two components, at the price of adding one more controller within the cluster. We use a CI-Jenkins/CD-Argo CD combination for our pipeline, which we think is more interesting.

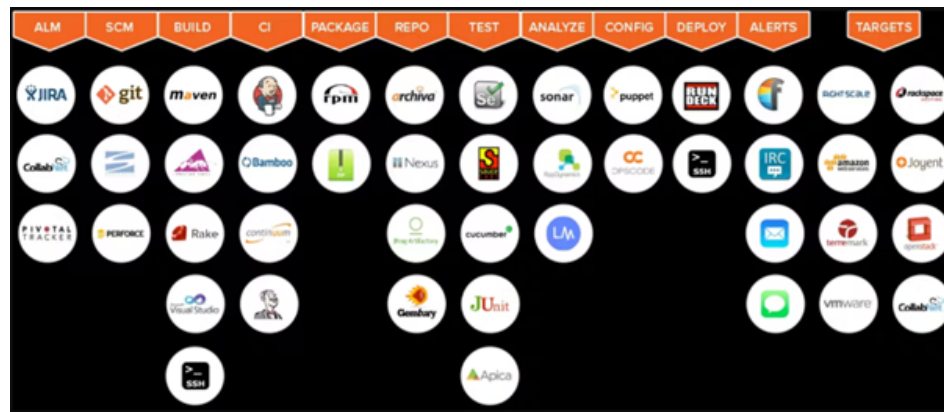


Figure 2.1: Comparison of existing CI/CD approaches

2.2 Research Gap

There has been a lot written about continuous integration, continuous deployment, containerisation, and orchestration. Usually, each area is discussed individually. The vendor documentation provides excellent information on how to configure each tool separately. Yet what is not abundantly available is a concrete example of tying these tools together and creating a viable pipeline for a real-world cloud infrastructure.

Our project is motivated by several concrete limitations that we faced while attempting to set up a CI/CD pipeline from scratch:

First of all, publicly available tutorials assume a triviality of an application to deploy. That is, an application has no native dependencies, does not use any machine learning models, and requires nothing special during building. This assumption is not very realistic even if one takes a look at our application's Dockerfile, deployment manifest, and runtime requirements.

Second of all, tutorials tend to get outdated. Tools like Jenkins, kubect!, eksctl, and Helm release new versions rather frequently and therefore procedures described in older tutorials cease to work with time. Repository signing keys expire. Repository locations change. Runtime versions required change too. Thus, a pipeline documenting these issues seems much more helpful than yet another tutorial.

Third: monitoring and observability become afterthoughts, either consigned to a different document or completely left out. We believed that Prometheus and Grafana should be integral to our system from the beginning and not something we tack onto our system later.

As illustrated in Fig. 2.2, current methodologies provide partial solutions but not a complete solution. The tutorials address individual tools. The vendor documents address individual products. The academic publications address individual practices.

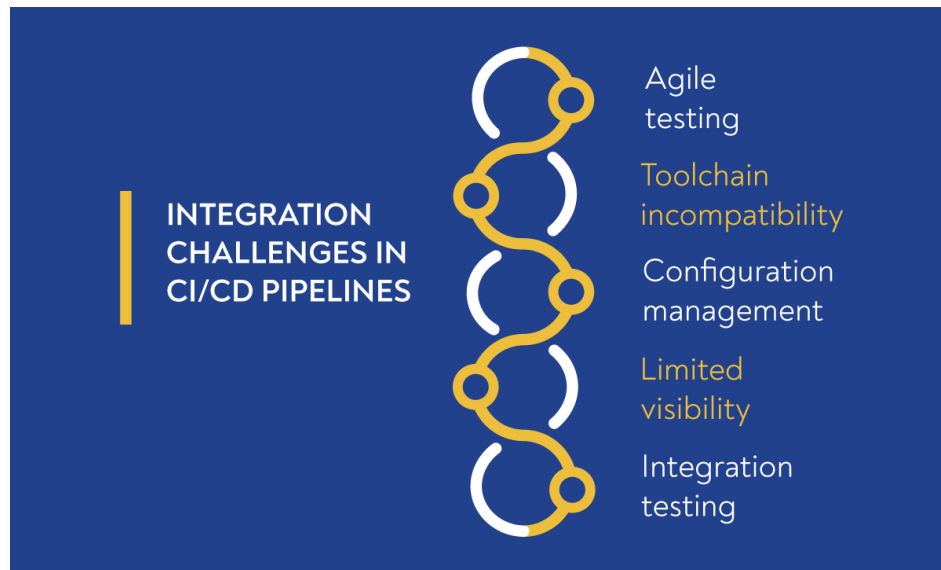


Figure 2.2: Identified research gap

Addressing all of that in a holistic manner and being upfront about the practicalities involved is what the gap is all about.

2.3 Objectives

What was our end game here? Very simple. A fully functional and documented CI/CD pipeline for our containerized ML application. One that is reproducible by one person. One that can be run on a modest cloud budget. And finally, one that follows industry best practices, not outdated tutorial guidelines.

Specifically, the objectives were:

1. **Create a Single-Ended Architecture:** The system has to integrate version control, continuous integration, container registry, container orchestration, and continuous deployment processes within a seamless pipeline which goes from a commit to a running deployment without any intervention.
2. **Proper Containerisation of the Application:** The Flask sentiment analysis application must be properly containerised, i.e., its Docker image should be built according to production-grade best practices like multi-stage building, not running as root, having all dependencies ready, etc. (Merkel, 2014)
3. **Create a Multibranch CI Pipeline:** The multibranch Jenkins pipeline has to be defined through the use of a Jenkinsfile. It should provide multibranch support with behaviour per branch, make use of the build number in image tagging, and handle the authentication tokens correctly.

4. **GitOps-Based Continuous Delivery:** Argo CD must be deployed inside the cluster and set up to monitor the Git repository, perform automated synchronization and auto-prune removal of resources as well as self-healing in case of drift (Weaveworks, 2017).
5. **Provide Reliable Cloud Infrastructure:** The infrastructure must be built on AWS with EKS orchestrator and a sufficient EC2-based managed node group that could accommodate the application replicas, its in-cluster controllers, and monitoring stack.
6. **Implement Observability from Day One:** A monitoring stack must be deployed along with the application, including cluster, node, and workload metrics collected via Prometheus and exposed via Grafana dashboards and Alertmanager alerts (Turnbull, 2018).
7. **Reveal the Truth about Operations:** The implementation must uncover and document all the operational challenges that arise in production deployment, including but not limited to expiring signing keys, runtime versions mismatch, credentials management peculiarities, and forgotten cloud resources cost.
8. **Test the Pipeline Fully:** The pipeline must be tested by completing multiple cycles of deployment to verify the ability to deploy commits to the cluster, perform zero-downtime rolling updates, monitor application and environment health accurately, and decommission the whole stack.

On the whole, this project aimed at proving that a relatively small development team or even a single developer can design and operate a complete continuous delivery platform, which would be virtually indistinguishable from the systems used by large-scale enterprises.

2.4 Problem Statement

Software engineering organizations today need to release features, bugfixes, and security updates continuously. All releases should be repeatable, reproducible, and reversible. In order to accomplish such feat, one needs an automated CI/CD pipeline that connects to the organization's version control system and its container repository, deploys to an orchestrator, and provides visibility into the live system for operators.

Such pipeline cannot be easy to build. It requires multiple tools, whose configurations are elaborate and complicated in their own right. But the hardest part comes

from interactions across these different tools. The problem becomes even more difficult to solve once we realize that the deployed application is going to be an ML-based service. As opposed to web applications, ML-based services have peculiar packaging requirements — they require models, libraries, corpora, etc.

There are a number of challenges inherent to the traditional way of deploying applications in CI/CD. In particular, the fact that the CI system must be trusted with credentials to production makes any breach of security in the CI system lead to direct production access by the attacker. It is impossible to keep track of what is currently running without discrepancies between code stored in version control and actual application code on the clusters due to no automated way to monitor such discrepancies and resolve them. Moreover, rollbacks tend to be manual.

However, a pull-based approach based on GitOps, where Git itself is considered to be the only source of the truth and an in-cluster controller automatically reconciles the current and desired states of application, does solve many problems. However, it does not solve the issue of image building, testing and pushing images, which still require CI functionality. Hence, the main challenge lies in designing a proper pipeline which would integrate both CI system and GitOps deployment along with running the containerised ML application on cloud infrastructure.

This project tackles all those aspects comprehensively — starting with the architecture, the actual implementation of the components involved, monitoring integration, operational challenges encountered during its course, and documenting everything in sufficient detail for reproduction or further development by another individual. The pipeline is designed for an environment with limited resources where AWS charges can be considered a genuine concern, therefore, teardown considerations have been taken into account.

2.5 Project Plan

It took place in a series of phases, with one building on another. At the outset, it became clear that an attempt to construct all this at once would result in trying to debug four items at once without knowing what was broken. Thus, the sequence outlined below is less formal than functional.

1. **Literature Review and Research:** The first stage was doing the reading. Here we reviewed the literature related to CI/CD, containerisation, container orchestration, and GitOps. The reading list contained classic papers on CI/CD practices, documentation for Jenkins, Docker, Kubernetes, and Argo CD, and numerous

practitioner’s blog posts and presentations about practical experience. The outcome of this stage was identifying the architectural solutions we would use as well as identifying gaps in the literature that we could help fill (Fowler, 2006; Humble and Farley, 2010; Burns et al., 2016).

2. **Pipeline Architecture Design:** In the second stage, the information gained from literature review translated into an architecture design for our pipeline. The major decisions taken during this stage include using Jenkins for CI, Argo CD for CD, Docker hub as a container registry, and Amazon EKS as an orchestration platform.
3. **Application Containerisation:** In phase three, we containerised the Flask-based sentiment analysis app into a Docker image using multi-staging of Dockerfile, resolving the native dependencies required by the openmp runtime used by XG-Boost, pre-downloading NLTK corpora so that the container would not need to download anything at runtime, and replacing the Flask development server with Gunicorn running as a non-root user.
4. **Cloud Infrastructure Provisioning:** In phase four, the cloud infrastructure was created using AWS. This involved creating the Jenkins EC2 instance, installing all the necessary software, assigning the correct IAM role to the instance, and creating an EKS cluster with `eksctl` along with a two-node managed node group of `t2-large` instances.
5. **CI Pipeline Implementation:** Phase five involved developing the Jenkins multi-branch pipeline which involved installing all the required plugins, providing access credentials for Docker Hub and GitHub, defining the CI pipeline consisting of three stages in the Jenkinsfile, and setting up the GitHub webhook for triggering builds.
6. **CD Pipeline Implementation:** Phase six consisted of the installation of Argo CD in the cluster and configuration of Argo CD for listening to the Git repository changes, applying the application definition file and enabling automatic sync.
7. **Integration of Observability:** In phase seven, the kube-prometheus-stack Helm chart was installed into a monitoring namespace, Grafana was accessible via a load balancer, and it was verified that the metrics were collected from the cluster and pods (Turnbull, 2018).
8. **Validation and Iterations:** The eighth phase involved deploying the pipeline several times and conducting tests to see how well rolling updates were performed, what effect configuration drift would have, and solving a number of operational challenges encountered throughout the process.

9. **Documentation and Report Generation:** In the final phase, a comprehensive report was compiled that documented the architecture, development, testing, and operational experience, providing sufficient details for further development or continuation by another developer.

Project Execution Timeline:

Project Execution Timeline:

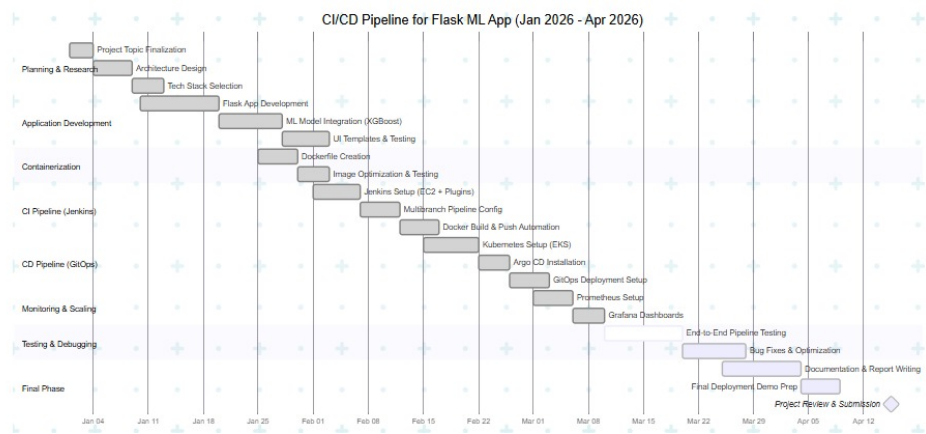


Figure 2.3: Project execution plan

Figure 2.3 shows a schematic overview of our planning and implementation. The timeline approximates when we actually carried out each phase, sometimes with overlap as appropriate for tasks which could be done concurrently. Nothing ever happened exactly as outlined—usually a bug in one phase would force us to go back to a previous phase—but our process held together.

The key to our success was breaking down our work into phases rather than attempting to implement all aspects simultaneously. We ended up with a functional pipeline along with a good understanding of how to make it work by destroying it several times during the process.

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 Technical Specifications

The objective of the project is to create a comprehensive CI/CD pipeline that automatically builds, packages, and deploys a containerized ML application into a managed Kubernetes cluster. Starting from a code commit into `main` to the delivery of a version into production, everything will be done without any manual interventions. Unlike common deployment pipelines that rely on manually triggered scripts or even push-based automation of the CI server, our solution leverages the GitOps pull approach, where an in-cluster controller continuously compares the current state of the cluster with the target state that is defined within version control. In order to validate if the architecture is feasible at all, we performed a requirements analysis that includes functional, non-functional, and a list of system requirements.

3.1.1 Functional Requirements

Functional Requirements describe what the system has to actually do:

- **Integration with Source Code Repository:** The solution must be integrated with the version control system, i.e., with GitHub and respond to certain events, such as commit and pull requests.
- **Automated Build of Application Image:** The software must automatically build an image of the specified application each time the code is committed into the main branch and apply a unique tag depending on the build number.
- **Upload of Image to Registry:** All successfully built images need to be uploaded by the system to the public registry, Docker Hub, with the credentials stored securely in the CI system, without being present anywhere else, including source.
- **Update of Manifest File and Commitment:** On upload of the new image, the solution has to update the corresponding deployment manifest file in the Git repo with the image tag and commit the change without triggering further builds.
- **Rolling Deployment:** The system must monitor the changes to deployment manifests and update them continuously by applying changes in a live cluster.

- **Zero-Downtime Rolling Update Deployment:** The deployment system must allow zero-downtime rolling update deployments wherein pods with a passing status replace the outdated pods after they pass the health check tests.
- **Health Checks:** The deployment process should continue with periodic monitoring of the pod status by carrying out health checks using liveness and readiness probes and then restarting pods that fail.
- **Drift Healing:** The deployment controller should detect manual drifts to the desired state of the cluster and take the required actions automatically to bring the actual state back to the desired state in the Git repository.
- **Metrics Gathering and Reporting:** Metrics should be continually gathered and reported through a dashboard visible to operators from within the cluster and the application.

3.1.2 Non-Functional Requirements

The Non-Functional Requirements are about how the system behaves rather than what it does:

- **Reliability:** The pipeline must be deterministic when executing for the same input data multiple times. Any commit should result in the exact same image built, manifest updated, and deployed version (Humble and Farley, 2010).
- **Security:** The CI system must not contain credentials for the production cluster. Credentials for external services, such as Docker Hub and GitHub, should reside only in the CI's internal credential store and should not be exposed to source code or build logs in any form. The container runtime must use an account that is not root.
- **Auditability:** Each deployment must trace back to one and only one commit. It should be possible to re-create the entire history of deployments by analyzing only the Git repository.
- **Cost Efficiency:** Infrastructure capacity should match the demand and allow clean teardown, so that unused resources do not incur costs. Cost of running a typical demonstration should remain below the cost of an average salary within a typical developer's budget.
- **Maintainability:** Pipeline setup must be declarative, allowing tracking via version control and code reviews.

- **Observability:** The condition and behavior of the system have to be observable by means of metrication, logging, and monitoring in order for the operator to understand what is happening without direct access to the underlying environment.
- **Reproducibility:** The entire pipeline, starting from the setting up of the environment to the deployment of the application, has to be reproducible based on the documentation provided.

3.1.3 System Requirements

The components needed for this architecture are:

- **Continuous Integration Server:** A Jenkins setup operating in an Ubuntu 24.04 EC2 instance of type t2.large, having Java 21, Docker, AWS CLI, kubectl, eksctl, and Helm installed. The EC2 instance will have 20 GB of EBS volume attached to it, along with an IAM role with full permissions to manage AWS resources.
- **Container Registry:** A Docker Hub registry the Jenkins instance can access via an access token that gives read-write-delete permissions to push images to the registry.
- **Source Repository:** A GitHub repository containing the source code for the application, Dockerfile, Jenkinsfile, Kubernetes manifests, and Argo CD app definition. This GitHub repository should have a webhook targeting the Jenkins CI server.
- **Container Orchestration Platform:** An Amazon EKS cluster with worker nodes provisioned in us-east-1 region, where a managed worker node pool contains two worker nodes of type t2.large. The cluster has to use the latest supported Kubernetes version provided by EKS.
- **GitOps Controller:** An Argo CD deployment running in the cluster in its own `argocd` namespace and available through a LoadBalancer service.
- **Monitoring Stack:** A kube-prometheus-stack Helm deployment running in the cluster in its own `monitoring` namespace and accessible using LoadBalancer services.
- **Pipeline Runtime Environment:** Python 3.11 with Flask, Gunicorn, NumPy, scikit-learn, XGBoost, and NLTK. The system is contained in a Docker container running on a Kubernetes Deployment with two replicas.

Table 3.1: System requirement specification

Component	Specification
CI Server	EC2 t2.large, Ubuntu 24.04, 20 GB EBS
Java Runtime	OpenJDK 21
CI Engine	Jenkins (latest stable, 2.541+)
Container Runtime	Docker (Ubuntu <code>docker.io</code> package)
Container Registry	Docker Hub (public repository)
Source Control	GitHub
Cluster Provisioning	eksctl (latest from eksctl-io)
Cluster Platform	Amazon EKS, us-east-1
Worker Nodes	2 × t2.large (managed node group)
GitOps Controller	Argo CD (stable manifests)
Monitoring	kube-prometheus-stack (Helm)
Application Language	Python 3.11
Web Framework	Flask 3.0.3
WSGI Server	Gunicorn 22.0.0
ML Libraries	XGBoost 2.1.1, scikit-learn 1.5.2, NLTK 3.9.1

All required hardware and software specifications are listed in Table 3.1.

This configuration provides us with adequate room for installing all necessary components - the Jenkins server, the Kubernetes control plane, the application pods, the GitOps controller, and monitoring infrastructure - within a single AWS account.

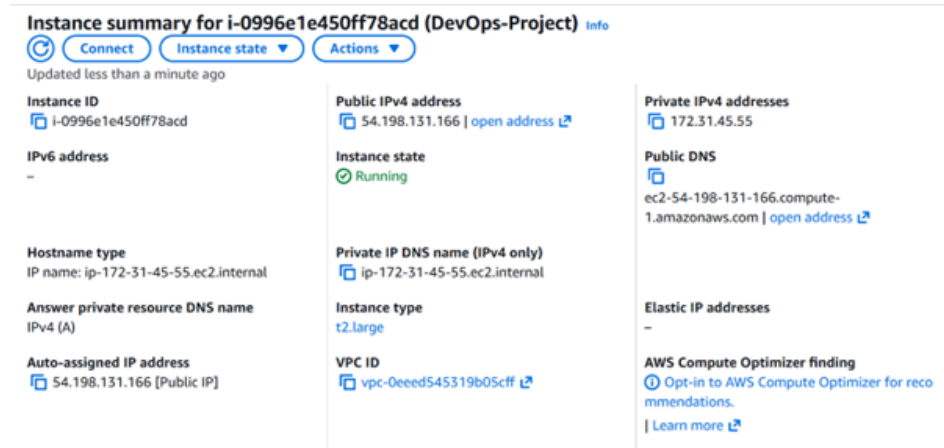


Figure 3.1: AWS infrastructure setup showing EC2 and EKS resources

Figure 3.1 illustrates the real deployment setup that was implemented for this work. The Jenkins EC2 instance acts as both the CI server and the host on which we initiate cluster commands, whereas the EKS cluster nodes execute application services, the Argo CD controller, and monitoring. In this regard, with the use of managed node groups, the worker node updates are taken care of by AWS.

3.2 Feasibility Study

This section walks through whether the pipeline is actually feasible, from three angles: technical, economic, and operational.

Technical Feasibility

All the technologies we need have proven themselves as being mature, extensively adopted, and well-documented. Jenkins has existed for almost fifteen years and remains the most popular CI tool in the market. Docker is the de facto standard technology for wrapping apps into containers. The orchestration technology we use — Kubernetes — is accessed via Amazon EKS and is the leading tool of its kind, adopted by all top cloud providers. ArgoCD, which is the GitOps controller technology, has become the most-popular open-source GitOps controller solution in the market and is used by numerous engineering companies (Burns et al., 2016; Weaveworks, 2017).

For the application side, adopting Python and Flask was a very straightforward decision. We don't require any training process to run our application because the models we employ are pre-trained. Thus, no special training facilities are required for running this system. Moreover, the pipeline we propose is modular and may be decomposed into multiple components.

Economic Feasibility

The cost to run the entire pipeline on AWS is fairly modest. A typical demo session of three hours or so runs for about two US dollars. Seven days of intense development cost somewhere in the 25–50 bracket, depending on our diligence in shutting everything down between demos (our diligence varied). The costs per hour and per day for each component are listed in Table 3.2.

This is easily within reach of a student budget, particularly with AWS credit offers around. The trick here is to be upfront about tearing everything down — and everything can be shut down between sessions — and this is documented in Chapter 6.

Operational Feasibility

The pipeline is set up in such a way that one engineer should be able to operate it. The pipeline is described with the help of stages along with the command lines that can be executed directly. The process of tearing down is also described similarly along with the verification stage to ensure that no services have been left running and are generating

Table 3.2: Cost profile of the deployed pipeline

Resource	Per hour (USD)	Per day (USD)
EC2 t2.large (Jenkins)	0.093	2.24
EKS control plane	0.100	2.40
2 × t2.large worker nodes	0.186	4.48
3 × Elastic Load Balancers	0.075	1.80
EBS and miscellaneous	—	0.50
Total	0.45	11.42

expenses without anyone noticing it.

Feasibility, informally, can be expressed as:

$$F = \frac{T + O}{C} \quad (3.1)$$

where T is technical feasibility, O is operational feasibility, and C is cost. A higher F means a more feasible system. For this pipeline all three factors come out positive — which is enough to say the project is doable at the proposed scale.

3.3 System Specification

The pipeline is made up of several modules that work together to take a developer commit and end up with a running production deployment.

- **Source Module:** All the source code, the configuration files, the manifests, and the application file used by Argo CD reside in the GitHub repo. This repo has a webhook that informs Jenkins of the latest commits.
- **CI Module:** The multibranch pipeline that resides in Jenkins is responsible for building, pushing, and updating the manifests. Jenkins takes care of compiling source code into a published container image and adding the tag to the repository.
- **Registry Module:** Docker Hub is the registry that contains the images which will be provided to the Kubernetes cluster on request. These images have build numbers that make reverting possible by updating the manifest file.
- **GitOps Module:** In Amazon EKS, there is an Argo CD controller that reconciles the Kubernetes cluster and the Git content. It is configured to automatically sync and self-heal if necessary.
- **Orchestration Module:** The Amazon EKS cluster is the location where the application workloads, Argo CD controller, and monitoring stack run. AWS is re-

sponsible for managing the cluster itself.

- **Monitoring Module:** In the release of kube-prometheus-stack, Prometheus, Grafana, and Alertmanager are deployed into the cluster. Metrics are collected from the cluster and application using Prometheus. The entire thing is visualized through Grafana using dashboards provided by the chart.

The separation of duties is maintained in the architecture. The CI pipeline creates artefacts alone. The CD pipeline deploys the artefacts alone. The monitoring pipeline monitors the environment alone. There are no privileged pieces that have access to end-to-end processes, which is beneficial from both security and failure scope perspectives.

Table 3.3: System modules and functionalities

Module	Functionality
Source Module	Version control and webhook trigger
CI Module	Image build, push, manifest update
Registry Module	Image storage and distribution
GitOps Module	Manifest reconciliation and self-healing
Orchestration Module	Workload scheduling and lifecycle management
Monitoring Module	Metrics collection and visualization

The architectural design consists of six layers – source, CI, registry, GitOps, orchestration, and monitoring. The source layer records commits by developers and notifies the CI layer via the webhook. The CI layer builds container images and publishes them, while at the same time updating the deployment manifest in the source layer. The GitOps layer detects changes to the manifest and triggers reconciliation in the orchestration layer. The monitoring layer keeps track of all activity in the orchestration layer and makes it visible via Grafana. This separation of concerns fits well with established best practices in cloud-native architecture design (Burns et al., 2016; Weaveworks, 2017).

3.4 Mathematical Model

A description of the pipeline’s dynamics is possible using just a few variables — one to denote time taken for each stage, and another to describe how much the executing system consumes. The total cycle time, which is defined as the time from commitment to production rollout, equals:

$$T_{total} = T_{ci} + T_{reg} + T_{poll} + T_{cd} \quad (3.2)$$

where T_{ci} is the time the Jenkins pipeline takes to build and push the image, T_{reg} is the time the registry takes to make the image available, T_{poll} is how long until Argo CD

next polls Git, and T_{cd} is the time Argo CD takes to apply the manifest and finish the rolling update.

The resource consumption of the running system can be written as a weighted sum over the application pods, the in-cluster controllers, and the monitoring stack:

$$R = \alpha C_{app} + \beta C_{ctrl} + \gamma C_{mon} \quad (3.3)$$

where C_{app} , C_{ctrl} , and C_{mon} are the CPU and memory usage of the application pods, controllers, and monitoring stack, and α , β , γ are weights reflecting how much each component contributes to the overall budget.

The goal of the deployment process is to keep T_{total} low while keeping R within what the cluster can handle. In practice, T_{ci} and T_{cd} dominate, and the registry latency T_{reg} is small enough to ignore.

3.4.1 Algorithm

The deployment process is implemented as a combination of the Jenkins pipeline and the Argo CD reconciliation loop. The high-level flow is below.

Algorithm 1 End-to-end deployment cycle

- 1: Developer commits code to main branch
 - 2: GitHub webhook triggers Jenkins
 - 3: Jenkins clones the repository
 - 4: Jenkins builds Docker image with tag `build-N`
 - 5: Jenkins pushes image to Docker Hub
 - 6: Jenkins updates `deployment.yml` with new tag
 - 7: Jenkins commits and pushes manifest with `[skip ci]`
 - 8: Argo CD polls Git repository at next interval
 - 9: Argo CD detects manifest change
 - 10: Argo CD applies new manifest to cluster
 - 11: Kubernetes performs rolling update of pods
 - 12: New pods pass readiness probes
 - 13: Old pods are terminated
 - 14: Service routes traffic exclusively to new pods
-

All of this is automated through and through and does not require any manual intervention from the point that the pipeline is established to apply the first Argo CD application.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The CI/CD pipeline is presented as a multi-layer architecture, consisting of source control, CI, container registry, GitOps deployment, container orchestration, and observability layers. The pipeline relies entirely on modern cloud-native concepts, and its multi-layer nature corresponds to the operational principle of separation of concerns — a separation of concerns that, in our experience, distinguishes a truly production-ready pipeline from a mere demonstration project (Burns et al., 2016; Weaveworks, 2017).

However, there is one particular point where the design of our architecture significantly diverges from a conventional push-type CI/CD pipeline. Indeed, the CI server of a push-type pipeline keeps credentials to the production cluster and deploys software updates to it after successfully building the package. Our architecture doesn't feature any such interaction between the CI server and the production cluster; once the update has been pushed to Git, the task of the CI server is completed, and everything further occurs within the cluster.

The architecture splits into six layers:

- **Source Control Layer:** The source of all truth concerning application code, build configuration, and cluster state lies within a GitHub repository. Any modifications in the environment will necessarily pass through the GitHub repo.
- **Continuous Integration Layer:** The continuous integration process includes building and packaging the application code in a multibranch Jenkins pipeline running on an EC2 instance. As output, the process will produce versioned Docker images and update the deployment manifest in the source control layer.
- **Registry Layer:** All images produced by the CI layer are stored in a public Docker Hub repo and delivered upon request by the Kubernetes cluster. Each image will be labeled with the Jenkins build number.
- **GitOps Layer:** An installation of Argo CD, running on the Kubernetes cluster, will continuously poll the source control layer and reconcile discrepancies between the current state of the cluster and the latest state defined in the deployment

manifest.

- **Orchestration Layer:** Amazon EKS cluster manages all application services, GitOps controller, and monitoring components. The AWS platform will manage the control plane of the cluster, while its nodes belong to a managed node group.

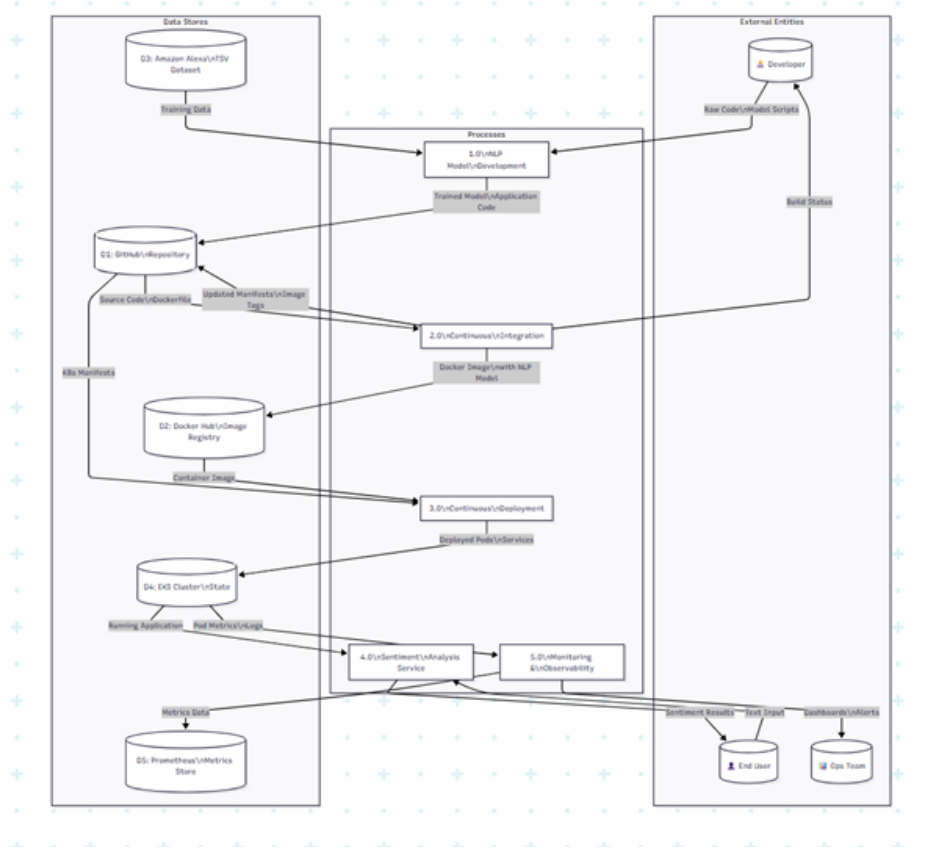


Figure 4.1: Overall system architecture

Architecture Representation:

How well the architecture works can be characterised by a relationship between automation coverage, operational simplicity, and security posture:

$$A = \frac{Auto + Sec}{Compl} \quad (4.1)$$

where *Auto* is how much of the pipeline is automated, *Sec* is the security posture, and *Compl* is the operational complexity. A higher *A* means a better architecture. Our pipeline scores high on both *Auto* and *Sec* while keeping *Compl* manageable for one engineer.

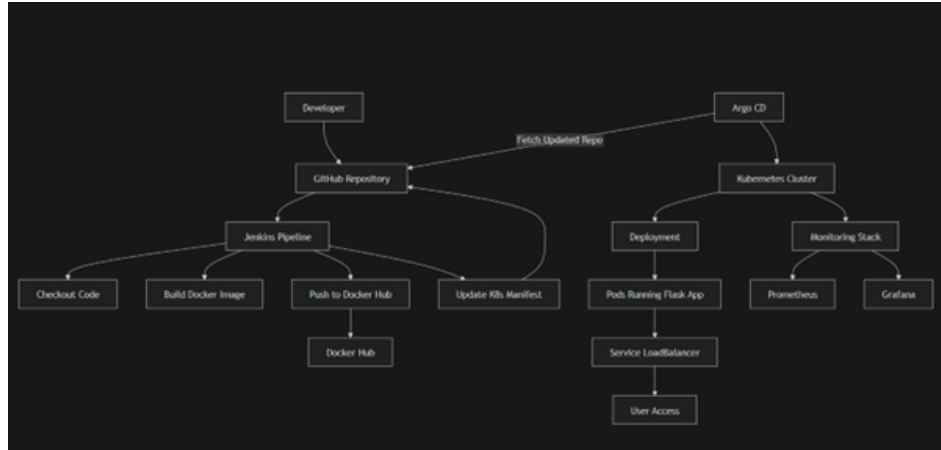


Figure 4.2: Detailed module interaction

Table 4.1: Architecture layer description

Layer	Function
Source Control Layer	Versioned storage of code and manifests
CI Layer	Build, test, and publish images
Registry Layer	Image storage and distribution
GitOps Layer	Continuous reconciliation from Git
Orchestration Layer	Workload scheduling and runtime
Observability Layer	Metrics, dashboards, and alerts

4.1.1 Continuous Integration with Jenkins Multibranch Pipeline

The Jenkins pipeline is defined using a Jenkinsfile, which is contained within the application repository. This was a conscious decision. Storing the definition of the pipeline in the application code means that any modification of the pipeline goes through the same pull request process as changes in the code itself, while also ensuring that the pipeline definition remains consistent with the code being built. There's no such thing as a separate "Jenkins configuration".

Using a multibranch pipeline enables Jenkins to create individual pipelines per branch within the repository. Special branching rules are enforced via the `when { branch 'main' }` statement, which limits the build and manifest steps only to the `main` branch. Feature branches receive a shallow checkout but not much else — no image is built from feature branches, no registry is polluted and inadvertent deployment from a WIP branch becomes impossible.

There are three stages in total:

1. **Checkout:** The repository is checked out in the Jenkins workspace. This step is executed for all branches.
2. **Build and Push Docker Image:** The Docker image is built from the Dockerfile

within the repository and is pushed to Docker Hub, using the `build-N` tag where `N` refers to the Jenkins build number. The authentication is provided via the `withCredentials` script which exports the Docker Hub username and token into the shell environment without logging them into the build console.

3. **Update Kubernetes Deployment YAML:** The image tag with file is updated by means of the `sed` command, changes are committed back to GitHub. The commit message contains the `[skip ci]` which avoids triggering a new Jenkins build from the auto-committed changes — otherwise we would end up in an infinite recursion of builds (we found that out the hard way when we went through thirty consecutive builds).
- [MINOR] Page Layout
Rule: Right margin overflow in LIST.
Expected: <= 523.276pt
Detected: 641.53pt
Confidence: 76.4%
Reason: Single line overflow.

4.1.2 GitOps Deployment with Argo CD

The GitOps controller responsible for doing the job on the continuous deployment side of the pipeline is the Argo CD. This tool is deployed within the EKS cluster, in an `argocd` namespace, and is available via the LoadBalancer service. The Argo CD Application refers to the application's Git repository and instructs Argo CD to monitor for any changes in the manifests present within the `kubernetes` folder.

The following are the three relevant behaviors configured in the Application:

- **Automatic Synchronization:** Changes made by Argo CD are automatically synchronized. No user pressing any kind of a sync button required. This is what really makes deployments unattended; otherwise, GitOps would simply become “your CI pushes changes to Git, but somebody has to still do the deployment.”
- **Cleanup of Deleted Resources:** Upon deletion of some resource from Git, Argo CD will delete it from Kubernetes too. Or else Kubernetes will end up with obsolete resources which nobody knows how they have appeared there.
- **Autorepair on Drift Detection:** Once a drift is detected (e.g. due to manual intervention in the cluster using `kubectl`), Argo CD corrects the situation making Kubernetes resources match those stored in Git. This is how Git is kept as the single source of truth all along the way.

Default Argo CD synchronization period is about 3 minutes. Which is quite enough to make deployments fast while avoiding unnecessary load on both clusters and git servers.

4.2 Design

This section describes both the architectural model as well as the interaction model, which dictates how the pipeline should be structured. The following subsections discuss the diagram that we used to determine data flow and boundaries for each subsystem within the pipeline.

4.2.1 Data Flow Diagram

The Data Flow Diagram (DFD) shows the flow of data through our pipeline starting from a commit performed by a developer to an executed pod. This is done in different levels of abstraction with additional internal details provided at each level.

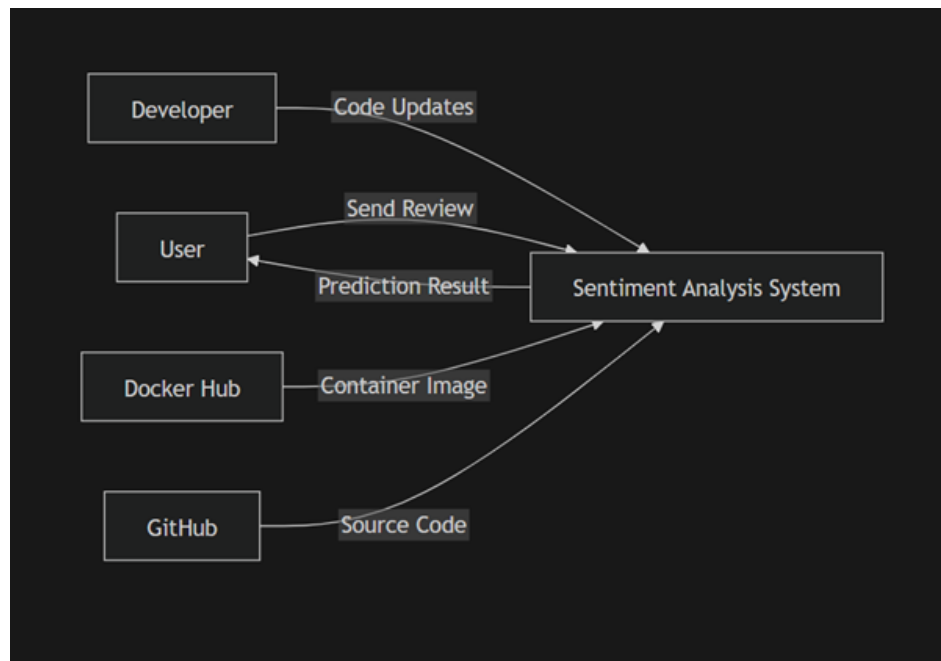


Figure 4.3: Level 0 – Context Diagram

- **Level 0 – Context Diagram:** In this diagram, which is called Level 0 Context Diagram, the entire system is depicted as one process, while the main external entities interacting with the system are mentioned. These include the Developer, who commits the code; the GitHub repository, where code and manifests reside; Jenkins server, where CI occurs; Docker Hub registry, which stores images; Argo CD controller that implements CD; EKS cluster that executes workloads; and the End User that utilizes the deployed application.
 - Developer commits the code on GitHub.
 - GitHub triggers Jenkins via a webhook.

- Jenkins builds and deploys the Docker image on Docker Hub as well as updates the manifest on GitHub.
 - Argo CD pulls the updated manifest on GitHub and applies it to the EKS cluster.
 - EKS cluster pulls the new image on Docker Hub and runs the updated application.
 - The End User accesses the application through the LoadBalancer URL.
- **Level 1 – Detailed Data Flow: Core Processes:** The Level 1 DFD subdivides the singular process described in Level 0 DFD into several processes forming the workflow as follows:
 - **Source Control:** Keeps code, configurations, and manifests; also emits webhook notifications on change events.
 - **Image Build:** Gets the code, builds the Docker image with a particular version and pushes it to the registry.
 - **Manifest Update:** Gets the image tag, modifies the deployment manifest, and pushes it to source control.
 - **Manifest Reconciliation:** Polls source control, detects any manifest changes, and invokes the orchestration API.
 - **Workload Execution:** Accepts the API invocation, pulls the images, and launches pods with applications.
 - **Level 2 – Detailed Processes:** This Level 2 DFD provides one additional level of detail for each subprocess.
 - **Image Build:** Clone the repo, build the Docker image using the Dockerfile, tag the image with the build number, log into Docker Hub, and push the image.
 - **Manifest Update:** Read the current manifest, perform the `sed` replacement, configure Git identity, commit the updated manifest, and push to GitHub with proper credentials.
 - **Manifest Reconciliation:** Poll the repo at intervals defined by configuration, compare the live manifest with Git manifest, detect manifest drift, and reconcile manifests upon changes.
 - **Workload Execution:** Receive new manifest, pull new Docker image, schedule pods, perform pod readiness checks, terminate pods, and update the service endpoint.

4.2.2 Use Case Diagram

Actors and Use Cases are mentioned in the Use Case Diagram.

- **Actors:**
 - **Developer:** Codes, commits, creates pull requests.
 - **Reviewer:** Reviews and approves pull requests.
 - **Jenkins Server:** Executes CI pipeline.
 - **Argo CD Controller:** Executes CD reconciliation process.
 - **Operator:** Monitors the system using Grafana and Argo CD interface.
 - **End User:** Utilizes the delivered application.
- **Use Cases:**
 - **Commit Code:** The developer pushes code changes to the server.
 - **Merge Pull Request:** The reviewer merges pull request in main.
 - **Build and Push Image:** Jenkins builds and deploys the Docker image.
 - **Update Manifest:** Jenkins pushes new image tag on Git.
 - **Reconcile Cluster:** Argo CD reconciles the cluster by pulling and applying new manifests.
 - **Monitor System:** The operator monitors performance metrics and the CI/CD pipeline.
 - **Access Application:** The end user sends a review and receives a sentiment analysis output.

4.2.3 Class Diagram

The Class Diagram shows the main logical entities involved in the pipeline and how they relate.

1. Classes:

- **Repository:**
 - Attributes: repoURL, branch, commitHash, webhookURL.
 - Methods: clone(), commit(), push(), receiveWebhook().
- **Pipeline:**

- Attributes: pipelineId, buildNumber, status, stages.
- Methods: checkout(), buildImage(), pushImage(), updateManifest().
- **Image:**
 - Attributes: imageName, tag, digest, size.
 - Methods: build(), tag(), push(), pull().
- **Manifest:**
 - Attributes: kind, name, namespace, imageReference, replicas.
 - Methods: read(), updateImageTag(), validate(), apply().
- **Application (Argo CD):**
 - Attributes: appName, repoURL, path, syncPolicy.
 - Methods: poll(), detectDrift(), reconcile(), prune().
- **Cluster:**
 - Attributes: clusterName, region, nodeGroup, namespaces.
 - Methods: schedulePod(), runProbe(), exposeService().

Key Functionalities:

- Source: Artifact management and webhook firing.
- Build pipeline: Image building and pushing.
- Image: Life cycle of container artifact.
- Manifest: State of cluster declaratively.
- App: GitOps for application.
- Cluster: Execution of workloads.

4.2.4 Sequence Diagram

The Sequence Diagram describes the interaction between the components in a standard deployment sequence.

1. Sequence Flow:

- **Commit:** The Developer invokes commit() and push() methods on the Repository.

- **Webhook Activation:** The Repository invokes `receiveWebhook()` method on the Jenkins Pipeline.
- **Image Creation:** The Pipeline invokes `buildImage()` and `pushImage()` methods on the Image, which in turn pushes it to Docker Hub.
- **Image Tag Update:** The Pipeline invokes `updateImageTag()` on the Manifest and updates the Repository.
- **Reconciliation:** The Argo CD Application invokes `poll()` on the Repository, detects the image update, and invokes `reconcile()` on the Cluster.
- **Workload Upgrade:** The Cluster invokes `schedulePod()`, `runsProbe()`, and `exposesService()`.

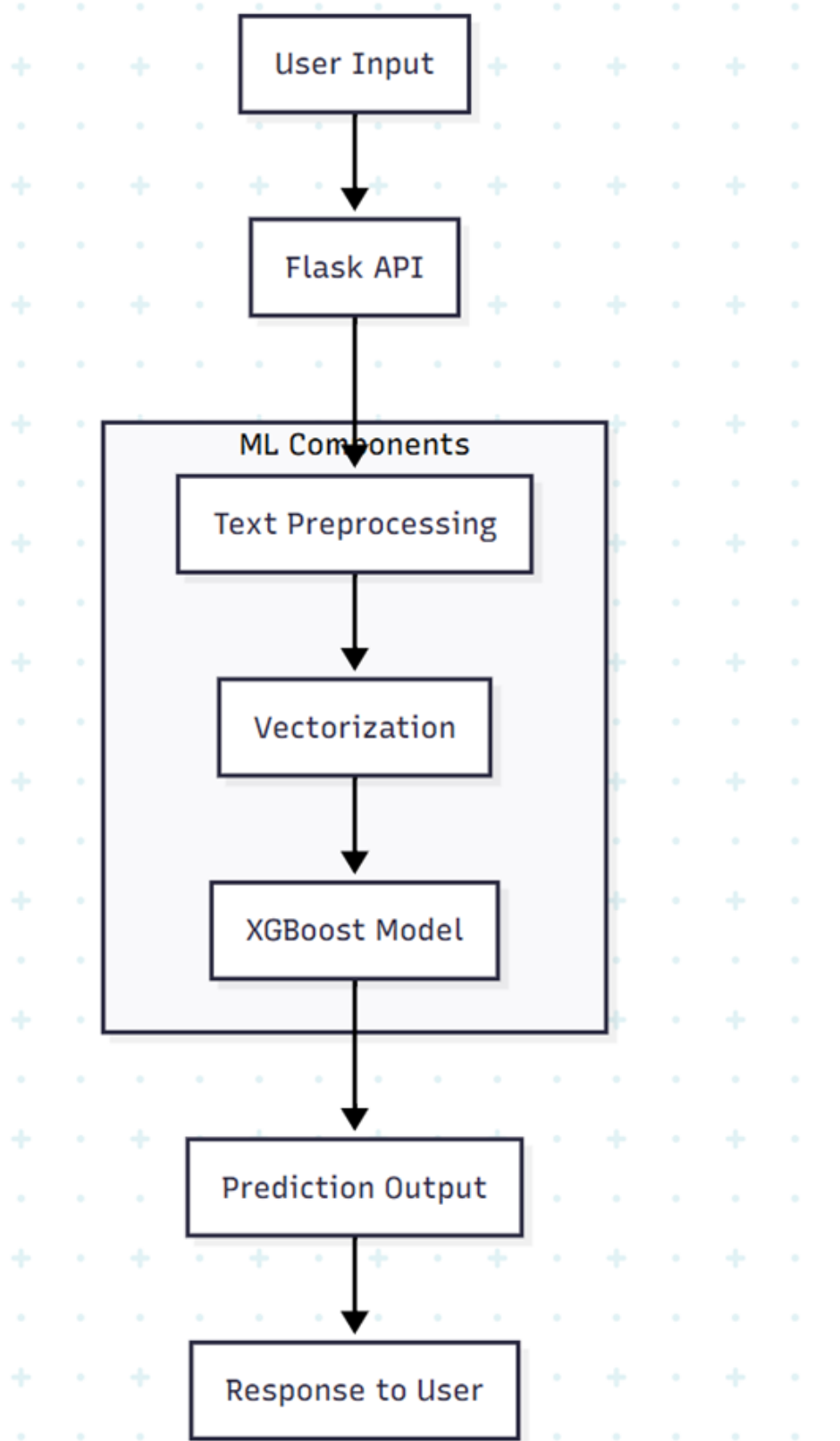


Figure 4.4: Level 1 – Detailed Data Flow: Core Processes

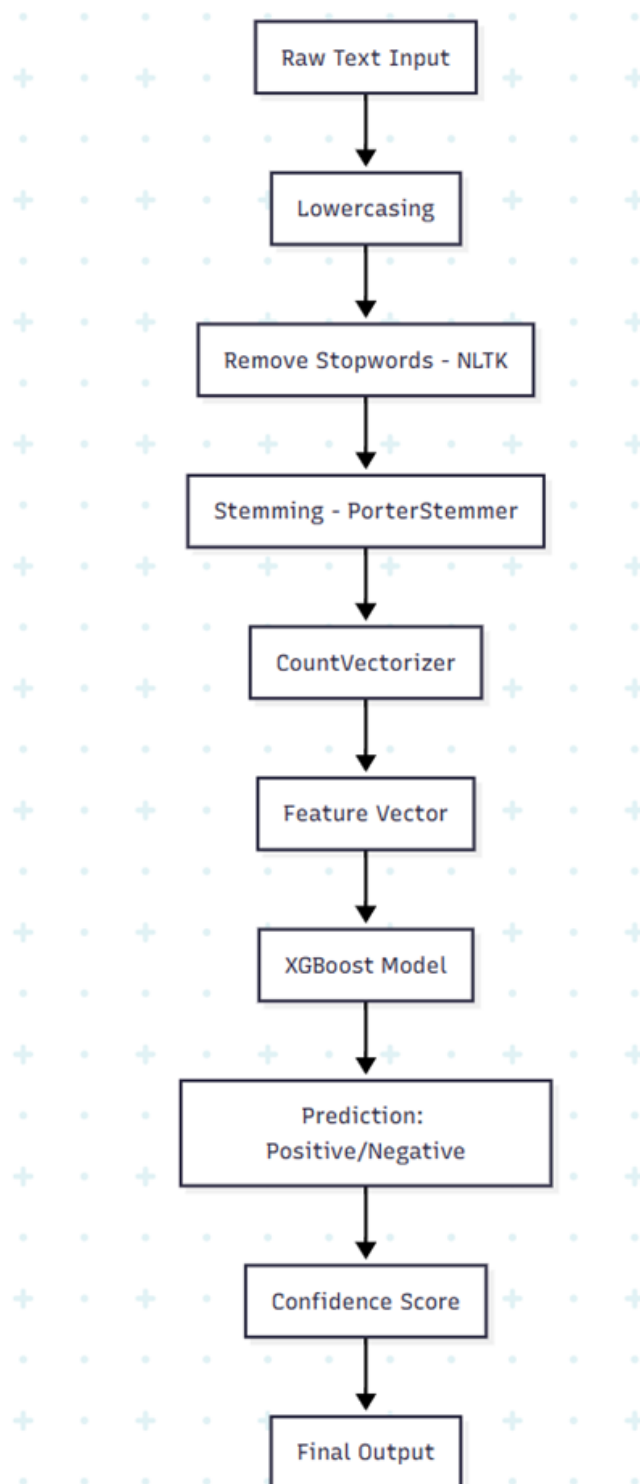


Figure 4.5: Level 2 – Detailed Processes within the Pipeline

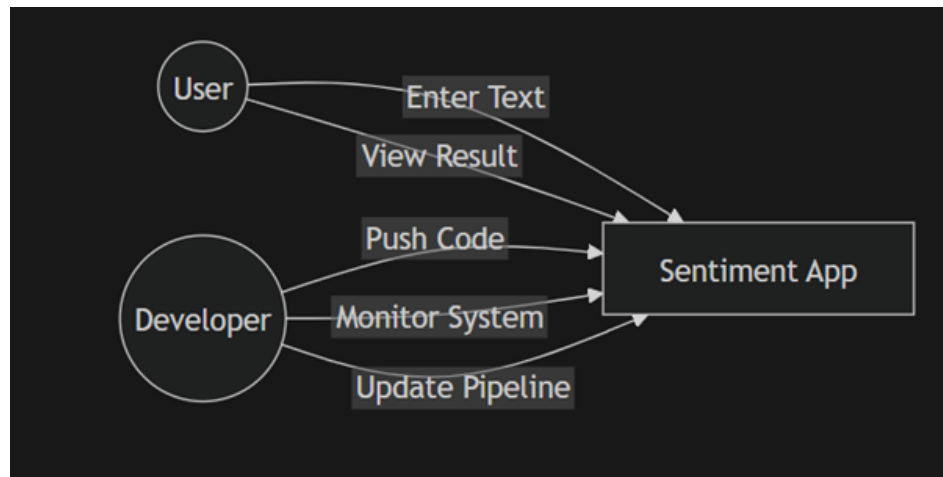


Figure 4.6: Use Case Diagram

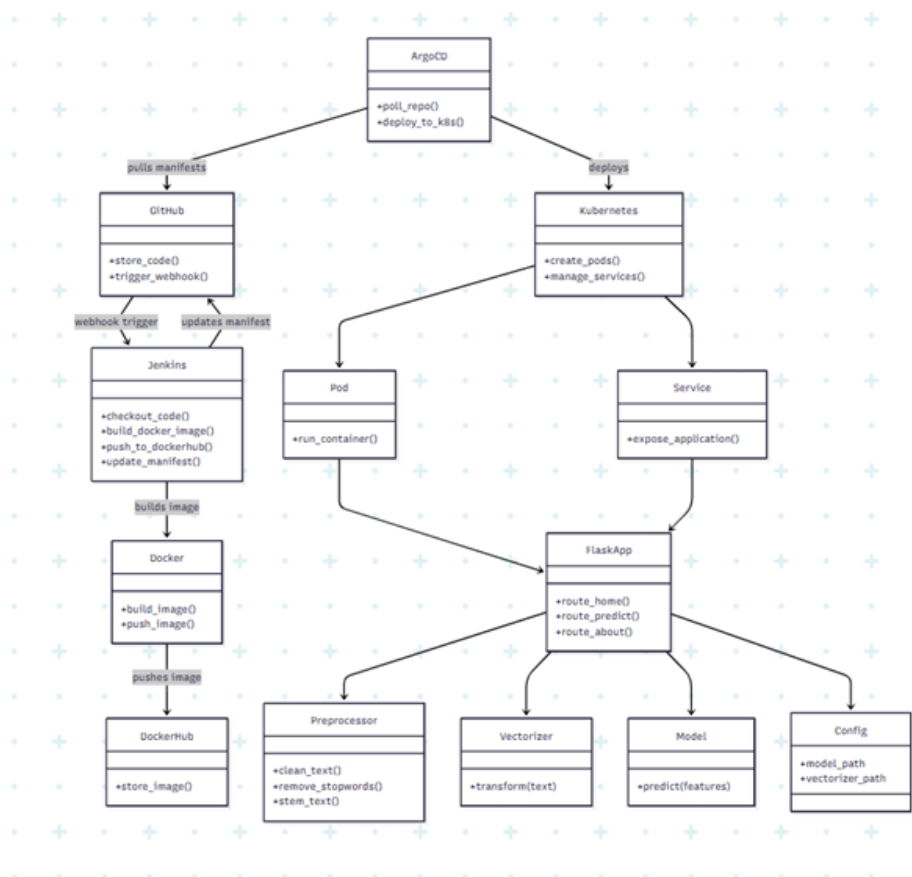


Figure 4.7: Class Diagram



Figure 4.8: Sequence Diagram

CHAPTER 5

METHODOLOGY AND TESTING

5.1 Overview

Construction and validation of the pipeline followed a series of well-defined phases which included: containerization of the application, setting up the infrastructure, creation of the CI pipeline, integration with GitOps for deployment, integration of observability, and validation. Each step was dependent on the other in sequence. The ultimate aim of having a fully automated and observable system which requires zero manual intervention was only feasible provided each phase stood true to its name.

5.1.1 Methodology

This process is covered in a number of steps, outlined below:

1. Application Containerisation:

- The containerised version of Flask sentiment analysis app uses multi-stage Dockerfile for packaging the app in Docker image.
- In the build stage, Python dependencies are installed along with preloading nltk data corpus into specific locations.

- The runtime stage installs only necessary runtime libraries for XGBoost.
- A non-root user is created during runtime and is then used to run the application.
- The application runs on Gunicorn instead of the standard development server provided by Flask framework.

[MINOR] Page Layout
Rule: Right margin overflow in LIST.
Expected: <= 523.276pt
Detected: 547.48pt
Confidence: 76.4%
Reason: Single line overflow.

2. Infrastructure Provisioning:

- An EC2 instance with sufficient processing capability is provisioned in order to deploy both the Jenkins server and the cluster orchestration tool.
- Necessary tools, i.e., Java 21, Docker, Jenkins, AWS CLI, kubectl, eksctl, and Helm are installed.

- Admin privileges are granted to an IAM role that is associated with the EC2 instance allowing management of AWS services.
- An EKS cluster is provisioned with eksctl with a managed worker group of size two with t2.large instance type.

3. CI Pipeline Implementation:

- The Jenkinsfile is added to the application repository and sets up a multi-branch pipeline with three stages.
- Credentials for Docker Hub and GitHub in Jenkins are set up with exact names used in the Jenkinsfile.
- A webhook is configured in GitHub that triggers builds whenever there is a new commit, enabling automatic pipeline execution.
- Branch-specific conditions ensure that new images are created and manifest updated only when committing to the main branch.

4. GitOps Deployment Integration:

- Argo CD is deployed to the Kubernetes cluster in its own namespace based on the official stable configuration and server-side apply.
- The Argo CD server is exposed using a LoadBalancer service for management access.
- Argo CD Application configuration is added, specifying the application repository along with auto-syncing, pruning, and self-healing.
- First-time synchronization is confirmed by observing Argo CD pulling the current manifest, creating the required Deployment/Service objects, and displaying healthy state.

5. Observability Configuration:

- The kube-prometheus-stack Helm chart is deployed in a separate monitoring namespace.
- Prometheus and Grafana are made LoadBalancer service types to access them from outside.
- The standard Grafana dashboards are reviewed to see whether they collect metrics from the cluster and the application pods.
- The initial Grafana admin password is extracted from the relevant Kubernetes secret.

6. End-to-End Verification:

- A dummy change in the code is pushed to the main branch.
- The triggering of the Jenkins job by the webhook is verified.
- The Docker image in Docker Hub is examined with the updated tag.
- The commit of the modified `kubernetes/deployment.yml` is verified in GitHub.
- It is verified that Argo CD detects the modification and deploys the application.
- The Kubernetes rolling update is tracked with `kubectl` and Argo CD interface.
- Finally, the updated application is tested from its LoadBalancer URL.

The end-to-end deployment cycle is captured by:

$$T_{cycle} = T_{ci} + T_{poll} + T_{cd} \quad (5.1)$$

where T_{ci} is the time the Jenkins pipeline takes, T_{poll} is the time before Argo CD's next reconciliation, and T_{cd} is the time taken to apply the manifest and finish the rolling update. In normal operation, T_{ci} runs two to three minutes, T_{poll} averages ninety seconds, and T_{cd} is around a minute. Total cycle time: about five minutes, commit to running deployment.

Algorithm 2 End-to-end pipeline workflow

- 1: Developer commits to main branch
 - 2: GitHub webhook triggers Jenkins
 - 3: Jenkins clones repository
 - 4: Jenkins builds Docker image with tag `build-N`
 - 5: Jenkins pushes image to Docker Hub
 - 6: Jenkins updates `deployment.yml` via `sed`
 - 7: Jenkins commits and pushes with `[skip ci]`
 - 8: Argo CD polls Git repository
 - 9: **if** manifest changed **then**
 - 10: Argo CD pulls new manifest
 - 11: Argo CD applies manifest to cluster
 - 12: Kubernetes performs rolling update
 - 13: New pods pass readiness probes
 - 14: Old pods are terminated
 - 15: Service routes traffic to new pods
-

Pipeline Flow

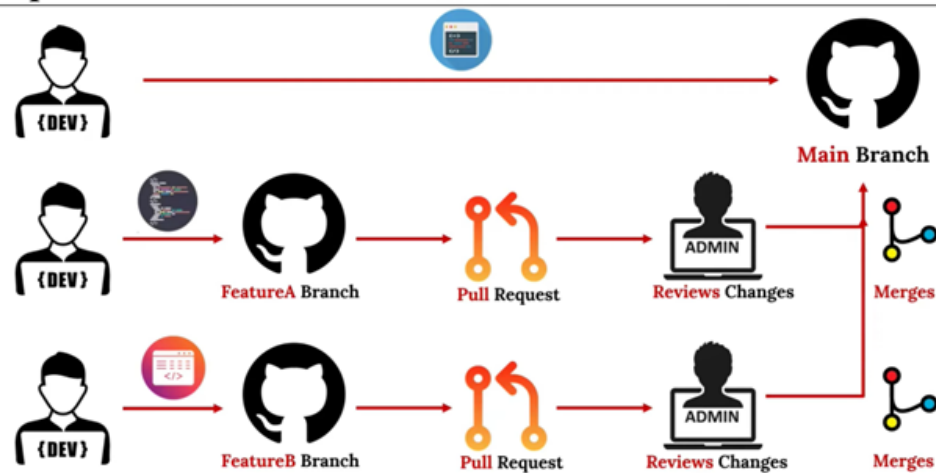


Figure 5.1: Methodology workflow of the proposed pipeline

5.2 Module Description

The system is divided into modules that have well-defined tasks. The division made development possible — whenever there was an error, we always had an idea which module to blame.

1. Source Control Module:

- Holds all the source codes of the application, including configuration files and Argo CD app definition.
- Triggers webhook upon commit to main branch.
- Validated for proper delivery of webhook and condition-based triggering based on branches.

2. CI Build Module:

- Executes Jenkins multi-branch pipeline.
- Resolves repository check out, image building, image pushing, and manifest update operations.
- Validated with regards to proper image tagging, registry image push, and auto-commit behavior.

3. Container Image Module:

- Packages up the application along with all of its required runtime dependencies.

- Comes with pre-baked NLTK corpora as well as OpenMP library, which is needed by XGBoost.
- Uses non-root privileges while keeping the final image small.
- Validated with regards to proper build process execution, container startup, and expected image size.

4. GitOps Deployment Module:

- Keeps on checking cluster configuration against the manifest within Git.
- Provides for an automated synchronization, pruning, and self-healing on drift.
- Validated in terms of manifest detection, rolling updates, and manual drift recovery.

5. Orchestration Module:

- Responsible for scheduling and launching application pods on the EKS cluster.
- Performs readiness checks and liveness checks, and does rolling updates.
- Testing involved pod scheduling, probe execution, and zero downtime rolling updates.

6. Observability Module:

- Fetches metrics from the cluster and application pods using Prometheus.
- Creates visual dashboards using the scraped metrics in Grafana.
- Tests included correct scraping of metrics, recent metrics in dashboards, and history of the metrics.

5.3 Testing Approach

The approach adopted for testing involved multiple layers based on the assumption that one layer would catch errors that were not caught by another layer.

1. Functionality Testing: All modules involved here are tested against their intended functionality. The Jenkins pipeline is tested by invoking a build and validating the correct output at each stage. Docker image is tested by invoking the image locally and ensuring that it responds to HTTP requests. Kubernetes deployment is tested by

ensuring the pods reach state Ready and successfully pass probes. Argo CD application is validated through observing it picking up manifest updates and applying them with the polling interval expected.

2. Integration Testing: Modules have to communicate with each other, and where they connect lies the toughest bugs. The GitHub webhook that triggers Jenkins' actions can be tested by making a commit and observing how Jenkins is triggered automatically. Jenkins' push to Docker Hub is tested by pulling and running the image somewhere else. The Jenkins auto-commit back to GitHub is checked by confirming the commit shows up in the repository and, more crucially, by verifying the `[skip ci]` marker prevents Jenkins from starting another build cycle.

3. System Testing: All steps are executed end-to-end. We commit changes to the source code and check the changes at each step in the system until the changes show up in the Argo CD's deployed application. We tried testing the success scenario as well as a few failure scenarios: failed Docker Hub push because of the revoked credentials, failed Jenkins commit because of the expired GitHub CD, and Argo CD sync failure due to the invalid manifest.

4. Performance Testing: Wall-clock times are recorded for each step in the process. The time taken for the Jenkins pipeline is recorded from start to finish. The time taken by the Argo CD polling is logged in the controller. Manifest commit to rolling update time and application response time under load are recorded separately. The latency can be seen on Grafana using a few runs of the sentiment analysis test.

5. Resilience Testing: Testing resilience means triggering a failure mode to see how it behaves. We kill a pod manually and check if the cluster replaces it. We manually edit a deployment manifest in the cluster using `kubectl`, which should be reverted automatically by Argo CD. We shut down the Jenkins server to test what happens to an in-progress build.

Table 5.1: Functional testing results

Module	Test Case	Status
Source Control Module	Webhook delivery	Passed
CI Build Module	Image build and tag	Passed
CI Build Module	Push to Docker Hub	Passed
CI Build Module	Manifest auto-commit	Passed
Container Image Module	Local container startup	Passed
GitOps Module	Manifest change detection	Passed
GitOps Module	Self-healing on drift	Passed
Orchestration Module	Rolling update	Passed
Orchestration Module	Readiness probe behaviour	Passed
Observability Module	Metric scraping	Passed
Observability Module	Dashboard rendering	Passed

CHAPTER 6

PROJECT IMPLEMENTATION

The rest of this chapter details the implementation of the CI/CD pipeline itself for the containerised sentiment analysis application. Topics include application packaging, infrastructure deployment, CI pipeline setup, GitOps deployment, and monitoring setup. There is sufficient detail provided in each section such that another engineer could replicate the entire solution with only this chapter and the source code snippets in Appendix A.

All components of the CI/CD pipeline operate on AWS. A single t2.large EC2 instance acts as the host server for Jenkins, while an Amazon EKS cluster made up of two t2.large instances acts as the orchestration platform. Importantly, even the CI pipeline configuration files are under version control along with the application source code, residing in the same GitHub repository — the Jenkinsfile, the application manifests and configurations, and the Argo CD application definition — so the CI process is audited like the rest of the code. This is in line with contemporary best practices (Burns et al., 2016; Weaveworks, 2017).

These components, which will be discussed in detail below, encompass containerization, infrastructure provisioning, continuous integration, continuous deployment with GitOps principles, observability, and lifecycle operations. All of these tasks are executed using their respective configuration files and documentation.

6.1 System Configuration

The system configuration configures the EC2 instance, EKS cluster, Jenkins server, and the necessary tools required for the operation of the pipeline process.

- **Set Up EC2 Instance:**

- An EC2 instance named “t2.large” is created from “us-east-1” region using the base image Ubuntu Server 24.04 LTS.
- Twenty GB of EBS volume is configured.
- Port 8080 is configured on the Security Group to provide access to the Jenkins Web Interface.

- The IAM role that has the policy “AdministratorAccess” is associated with the EC2 instance to configure AWS resources using AWS CLI and eksctl.
- **Tooling Installation:**
 - Docker is installed using the Ubuntu `docker.io` package and configured to start at boot.
 - OpenJDK 21 is installed because the current Jenkins release needs Java 21 and refuses to start under Java 17.
 - Jenkins is installed from the official Debian-stable repo, with the 2026 GPG signing key, because the 2023 key expired in March 2026 and breaks every tutorial that has not been updated.
 - AWS CLI version 2 is installed from the official Amazon zip distribution.
 - `kubectl` is installed using the latest stable version from `dl.k8s.io`.
 - `eksctl` is installed from the `eksctl-io` GitHub release, since the older Weaveworks repo has been deprecated.
 - Helm is installed using the official installation script.
- **Cluster Setup:**
 - The EKS cluster, named `sentiment-cluster`, is created using `eksctl` with a managed node group of two `t2.large` workers.
 - The cluster is set up in `us-east-1` with default networking.
 - Cluster connectivity is checked using `kubectl get nodes`.

6.2 Feature Demonstration

This section runs through the main features of the implemented pipeline.

1. Automated Build on Commit:

- Every commit to main fires an automatic build through the GitHub web-hook.
- The build produces a uniquely tagged Docker image with no manual intervention.

2. Versioned Image Tagging:

- Every image is tagged with the Jenkins build number, so rolling back to any previous version just means editing the manifest.

- The image tag is recorded both on Docker Hub and in the Git history.

3. **GitOps-Based Deployment:**

- Argo CD constantly reconciles the cluster state with the manifest in Git.
- Operators do not run `kubectl apply` by hand; the cluster always matches whatever is currently in Git.

4. **Self-Healing on Drift:**

- If a deployment is changed by hand through `kubectl`, Argo CD spots the drift and pulls the cluster back to match Git.
- This means Git stays the single source of truth.

5. **Continuous Observability:**

- Prometheus constantly scrapes metrics from the cluster and the application pods.
- Grafana provides ready-made dashboards for cluster health, per-namespace resource usage, per-node metrics, and network throughput.

6.3 **Testing Scenarios**

We exercised the pipeline under several test scenarios to check how it behaves under both normal and unexpected conditions.

1. **Normal Operation:**

- A code change is committed to main, the pipeline runs through to completion, and the new version is deployed automatically.
- The total cycle time from commit to running deployment comes out to about five minutes.

2. **Feature Branch Behaviour:**

- A commit on a feature branch triggers the Checkout stage but skips Build and Push and Update Manifest.
- We checked this by confirming that no new image was pushed to Docker Hub and no commit was made to the manifest.

3. **Drift Recovery:**

- The replica count in the running deployment is changed by hand using `kubectl`.
- Argo CD spots the drift within the polling interval and resets the replica count to the value in Git.

4. **Rollback:**

- The image tag in `deployment.yml` is rolled back to a previous build number.
- Argo CD picks up the change and rolls the deployment back to the previous version.
- The application keeps serving traffic the whole time.

5. **Failure Recovery:**

- A pod is deleted by hand using `kubectl`.
- The Kubernetes deployment controller schedules a replacement pod automatically.
- Service traffic gets routed to the surviving pod during the restart, which confirms zero-downtime behaviour.

6.4 Expected Outcomes

The expected outcomes of the implementation are:

1. **Reliable Automated Delivery:**

- Every commit to main produces a deployed version on the cluster.
- Failures are surfaced through clear status reporting in Jenkins, Argo CD, and Grafana.

2. **Auditable Deployment History:**

- The Git history records every change to the deployment manifest.
- The state of the cluster at any point in time can be reconstructed from Git alone.

3. **Operational Self-Service:**

- Operators do not need direct cluster access to trigger deployments.
- All deployments go through normal Git workflows.

4. Observable System Behaviour:

- The state of the pipeline, the cluster, and the application is visible through dashboards.
- Metric history survives pod restarts.

6.5 Application Containerisation

The first concrete step of the implementation is producing a Docker image that wraps the Flask sentiment analysis app along with everything it needs to run. The image is built through a multi-stage Dockerfile, which separates build-time concerns from runtime concerns and gives us a smaller, more secure final image (Merkel, 2014).

The build stage installs Python dependencies into a user-local site-packages directory and pre-downloads the NLTK stopwords corpus to a fixed location. The reason for pre-downloading is straightforward: a running container shouldn't have to make a network call to nltk.org at startup just to fetch a corpus. If we left that out, our app's availability would silently be tied to whether some download server somewhere was reachable.

```
1 # ---- Stage 1: Builder ----
2 FROM python:3.11-slim AS builder
3 WORKDIR /app
4 RUN apt-get update && apt-get install -y --no-install-recommends \
5     build-essential \
6     && rm -rf /var/lib/apt/lists/*
7 COPY requirements.txt .
8 RUN pip install --no-cache-dir --user -r requirements.txt
9 RUN python -m nltk.downloader -d /usr/local/nltk_data stopwords
10
11 # ---- Stage 2: Runtime ----
12 FROM python:3.11-slim
13 WORKDIR /app
14 RUN apt-get update && apt-get install -y --no-install-recommends \
15     libgomp1 \
16     && rm -rf /var/lib/apt/lists/* \
17     && useradd --create-home --shell /bin/bash appuser
18 COPY --from=builder /root/.local /home/appuser/.local
19 COPY --from=builder /usr/local/nltk_data /usr/local/nltk_data
20 COPY --chown=appuser:appuser app.py config.py ./
21 COPY --chown=appuser:appuser models/ ./models/
22 COPY --chown=appuser:appuser templates/ ./templates/
23 COPY --chown=appuser:appuser static/ ./static/
24 USER appuser
```

```

25 ENV PATH=/home/appuser/.local/bin:$PATH \
26     PYTHONDONTWRITEBYTECODE=1 \
27     PYTHONUNBUFFERED=1 \
28     NLTK_DATA=/usr/local/nltk_data \
29     FLASK_APP=app.py
30 EXPOSE 5000
31 CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", \
32     "--threads", "4", "--timeout", "60", "app:app"]

```

Listing 6.1: Multi-stage Dockerfile

A few decisions in this Dockerfile move it past tutorial-grade. Using `appuser` as the base keeps the final image small — a few hundred megabytes instead of a gigabyte. The `libgomp1` package goes in the runtime stage because it depends on the OpenMP runtime to load, and without it the app just dies at startup with a confusing import error. We learned this the hard way — it was probably an hour of debugging before either of us thought to check whether XGBoost was even managing to import. The app runs as a non-root user (`appuser`). Basic, but important. And the app launches through Gunicorn with two worker processes and four threads per worker, which gives reasonable concurrency for low-to-moderate request volumes.

[MINOR] Page Layout
Rule: Right margin overflow in BODY_TEXT.
Expected: <= 523.276pt
Detected: 562.93pt
Confidence: 77.7%
Reason: Single line overflow.

Tag	Digest	OS/ARCH	Compressed size
build-5 Last pushed 34 minutes by rajrishidockerhub	83c273d93e4f	linux/amd64	585.32 MB
build-3 Last pushed about 3 hours by rajrishidockerhub	9e97ac5c5501	linux/amd64	585.32 MB

Figure 6.1: Docker image build output

6.6 Jenkins Pipeline Implementation

The CI side of the pipeline is a Jenkins multibranch pipeline, driven by the `Jenkinsfile` at the root of the application repo. Three stages. Two sets of Jenkins credentials — one for Docker Hub, one for GitHub.

```

1 pipeline {
2     agent any
3     options { disableConcurrentBuilds() }
4     environment {
5         IMAGE_NAME = "rajrishidockerhub/sentiment-app"

```

```

6     GIT_USER    = "johnthebrat7"
7     GIT_EMAIL   = "rajrishisamanta2@gmail.com"
8     GIT_REPO    = "johnthebrat7/Corporate-CI-CD-pipeline"
9
10    stages {
11        stage('Checkout') { steps { checkout scm } }
12        stage('Build and Push Image') {
13            when { branch 'main' }
14            steps {
15                script {
16                    env.IMAGE_TAG = "build-${BUILD_NUMBER}"
17                    withCredentials([usernamePassword(
18                        credentialsId: 'dockerhub-creds',
19                        usernameVariable: 'DOCKER_USER',
20                        passwordVariable: 'DOCKER_PASS'
21                    )]) {
22                        sh """
23                            docker build -t ${IMAGE_NAME}:${IMAGE_TAG} .
24                            echo "\$DOCKER_PASS" | docker login -u "\
$DOCKER_USER" --password-stdin
25                            docker push ${IMAGE_NAME}:${IMAGE_TAG}
26                            """
27                    }
28                }
29            }
30        }
31        stage('Update K8s Manifest') {
32            when { branch 'main' }
33            steps {
34                script {
35                    withCredentials([usernamePassword(
36                        credentialsId: 'github-creds',
37                        usernameVariable: 'GIT_USERNAME',
38                        passwordVariable: 'GIT_TOKEN'
39                    )]) {
40                        sh """
41                            set -e
42                            git config user.name "${GIT_USER}"
43                            git config user.email "${GIT_EMAIL}"
44                            git fetch origin
45                            git checkout main
46                            git reset --hard origin/main
47                            sed -i "s|image:.*|image: ${IMAGE_NAME}:${{
IMAGE_TAG}}|" \
48                                kubernetes/deployment.yml
49                            git add kubernetes/deployment.yml
50                            git diff --cached --quiet || git commit -m \

```

```

51         "ci: update image to ${IMAGE_TAG} [skip
ci]"
52         git push https://\${GIT_USERNAME}:\
\${GIT_TOKEN}@github.com/\${GIT_REPO}.git main
53         ""
54     }
55 }
56 }
57 }
58 }
59 post { always { sh 'docker image prune -f || true' } }
60 }

```

Listing 6.2: Jenkinsfile

The Jenkinsfile leans on a few patterns common to real-world pipe

[MINOR] Page Layout
Rule: Right margin overflow in BODY_TEXT.
Expected: <= 523.276pt
Detected: 665.96pt
Confidence: 77.7%
Reason: Single line overflow.

option stops two simultaneous builds of the same branch from racing the Docker push and Git commit stages. The `when { branch` tive limits image production to commits on `main`, so feature-branch experimentation doesn't pollute the registry. The `[skip ci]` marker in the auto-commit message keeps Jenkins from treating its own commits as triggers — without it, every CI run would trigger another CI run, and Jenkins would happily eat itself in an infinite loop. We discovered this on our first try and burned through about thirty consecutive builds before noticing what was happening. The `post { always }` block runs Docker image pruning at the end of every build, which keeps the Jenkins server's disk from filling up with stale layers.

	Checkout SCM	Checkout	Push Image	Manifest	Post Actions
Average stage times: (full run time: ~4min 52s)	779ms	696ms	3min 47s	1s	1s
#5 Apr 27 10:51 No Changes	677ms	693ms	4min 43s	1s	1s
#4 Apr 27 10:29 2 commits	833ms	641ms	1s failed	87ms failed	335ms
#3 Apr 27 08:51 4 commits	794ms	666ms	4min 44s	1s	906ms
#2 Apr 23 14:17 No Changes	710ms	756ms	4min 45s	1s	1s
#1 Apr 23 13:48 No Changes	884ms	724ms	4min 44s	762ms	1s

Figure 6.2: Jenkins multibranch pipeline view

6.7 Kubernetes Manifest Implementation

The Kubernetes manifests are where the runtime state we want for the application is actually described. They live in the `kubernetes/` directory of the repo — a Deployment manifest and a Service manifest.

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: sentiment-app
5   namespace: default
6   labels:
7     app: sentiment-app
8 spec:
9   replicas: 2
10  selector:
11    matchLabels:
12      app: sentiment-app
13  template:
14    metadata:
15      labels:
16        app: sentiment-app
17    spec:
18      containers:
19        - name: sentiment-app
20          image: rajrishidockerhub/sentiment-app:build-1
21          ports:
22            - containerPort: 5000
23          env:
24            - name: DEBUG
25              value: "False"
26          resources:
27            requests:
28              memory: "256Mi"
29              cpu: "200m"
30            limits:
31              memory: "512Mi"
32              cpu: "500m"
33          readinessProbe:
34            httpGet: { path: /, port: 5000 }
35            initialDelaySeconds: 10
36            periodSeconds: 5
37          livenessProbe:
38            httpGet: { path: /, port: 5000 }
39            initialDelaySeconds: 30
40            periodSeconds: 15
```

Listing 6.3: Kubernetes Deployment manifest

Replicas, two purposefully. Sufficient redundancy to support rolling updates without building up resource utilization. Initially, the container image will refer to the `build-1` tag, but Jenkins will update it for each main branch build using the `sed` command in the third step of the pipeline. The `resources` section defines both request and limit settings requests to tell the scheduler where to schedule pods, and limits to prevent an out-of-control pod from consuming all resources of the node. Readiness and liveness probes allow the cluster to distinguish between a pod initializing, a healthy pod, and one in a catatonic state requiring replacement.

```
1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: sentiment-app-service
5   namespace: default
6 spec:
7   type: LoadBalancer
8   selector:
9     app: sentiment-app
10  ports:
11    - name: http
12      port: 80
13      targetPort: 5000
14      protocol: TCP
```

Listing 6.4: Kubernetes Service manifest

The manifest file for the Service routes the application behind the AWS Elastic Load Balancer. Incoming traffic reaches port 80 of the load balancer, which directs it to port 5000 of the application's pods. The service selector will match the labels assigned to the pods by the Deployment, hence ensuring that any updates will be reflected in the pods without manual intervention.

6.8 Argo CD Application Implementation

The Argo CD itself gets deployed within the cluster via upstream stable manifests with server-side apply. The use of server-side apply is necessary in this case because the size of the Argo CD CRDs is too big for the regular `last-applied-configuration` annotation constraint. We found out about it the hard way because we tried deploying it using client-side apply and ended up running into an annotation size limit error.

The Argo CD Application manifest is the one responsible for establishing the connection between the Git repository and the cluster.

```
1 apiVersion: argoproj.io/v1alpha1
2 kind: Application
3 metadata:
4   name: sentiment-app
5   namespace: argocd
6 spec:
7   project: default
8   source:
9     repoURL: https://github.com/johnthebrat7/Corporate-CI-CD-pipeline
10    .git
11    targetRevision: main
12    path: kubernetes
13 destination:
14   server: https://kubernetes.default.svc
15   namespace: default
16 syncPolicy:
17   automated:
18     prune: true
19     selfHeal: true
20   syncOptions:
21     - CreateNamespace=true
```

Listing 6.5: Argo CD Application definition

The `syncPolicy.automated` configuration snippet ensures the process of synchronization will be continuous, without requiring any manual intervention like syncing via button push. The `prune: true` flag ensures that resources deleted from Git will be deleted from the cluster as well. This avoids the situation of accumulation of unnecessary orphaned resources within the cluster. The flag `selfHeal: true` guarantees that any discrepancy between Git and cluster will be sorted out automatically.

6.9 Observability Implementation

The observability stack is installed via the `kube-prometheus-stack` Helm chart, which is basically a collection of all the necessary components, including Prometheus, Grafana, Alertmanager, and their respective CRDs and exporters. The Helm chart is deployed to its own `monitoring` namespace, and Prometheus and Grafana are configured as LoadBalancer services, allowing us to access them externally (Turnbull, 2018).

Installation is just:

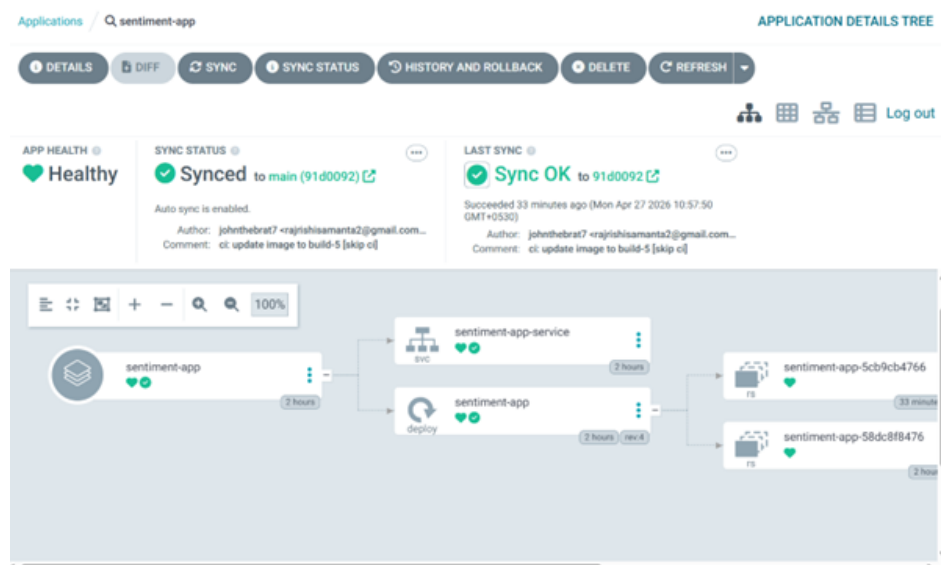


Figure 6.3: Argo CD application in synced and healthy state

```

1 helm repo add prometheus-community \
2   https://prometheus-community.github.io/helm-charts
3 helm repo update
4 kubectl create namespace monitoring
5 helm install monitoring prometheus-community/kube-prometheus-stack \
6   --namespace monitoring \
7   --set prometheus.service.type=LoadBalancer \
8   --set grafana.service.type=LoadBalancer

```

Listing 6.6: Helm installation of the monitoring stack

Once installed, the Grafana admin password is retrieved from the corresponding Kubernetes secret (which is auto-generated, and the Grafana stores it in a secret that corresponds to the name of the Helm release). The Grafana UI can be accessed via the LoadBalancer URL, while the built-in dashboards provide information on the cluster status, resource usage per namespace, CPU/Memory usage per node, and network bandwidth. It is worth mentioning that designing the dashboards ourselves took considerable effort.

6.10 Implementation Summary

The implementation ties up the source control, continuous integration, container registry, GitOps delivery, container orchestration, and monitoring into one cohesive process chain. The modular approach is what makes each component independently betterable or changeable in any way possible — along with the fact that all of the processes are declarative, which means it can be recreated based on only the files stored in the

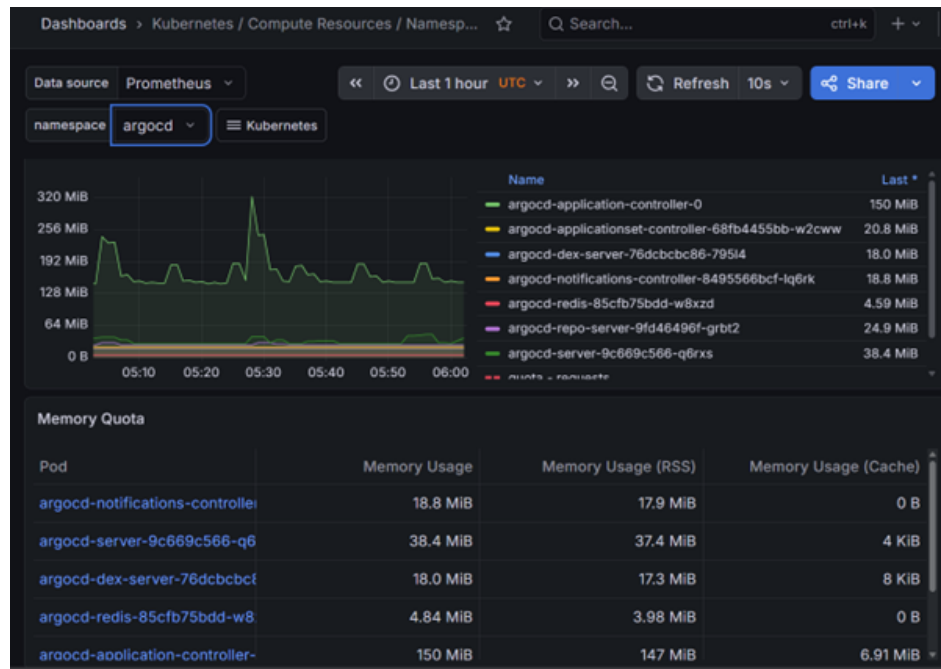


Figure 6.4: Grafana dashboard showing cluster metrics

repository.

Table 6.1: Implementation module summary

Module	Implementation Artefact
Source Control	GitHub repository
Container Image	Dockerfile
CI Pipeline	Jenkinsfile
Deployment Manifest	kubernetes/deployment.yml
Service Manifest	kubernetes/service.yml
GitOps Application	argocd/application.yml
Observability	kube-prometheus-stack Helm release

CHAPTER 7

RESULTS AND DISCUSSION

We tested our CI/CD pipeline on the live AWS environment discussed in earlier chapters. Briefly, what emerged was an entire system capable of delivering the selected application by means of fully automated end-to-end delivery, leveraging the selected combination of Jenkins, Docker, Amazon EKS, Argo CD, and kube-prometheus-stack.

The pipeline was evaluated according to five practical qualities — total cycle time for committing changes and deploying them, resources consumption of the in-cluster components, deployability across multiple runs, drift recovery capabilities, and operational costs. Our observations matched well with the advantages reported in the current literature on cloud-native software engineering for the application of GitOps (Burns et al., 2016; Weaveworks, 2017; Sharma, 2017). Indeed, the integration between a well-established CI framework and a pull deployment control engine enables the development of an end-to-end delivery platform which is responsive and easy to manage. No sorcery here. Just good practices and smart choices.

Our findings demonstrate that the selected architectural solution represents a definite upgrade over traditional push deployment strategies on most relevant dimensions.

7.1 Experimental Results

The pipeline was validated by going through full cycles of deployment. In each cycle, a code update was committed to `main`, and the process terminated when the updated application was successfully serving traffic from the EKS cluster. In the entire validation effort, all commits went from Jenkins through Docker Hub, GitHub, and Argo CD with no human intervention at any point.

Stage by stage, the pipeline worked as expected. The `Checkout` stage completed within seconds. The `Build` and `Push Image` stage took two or three minutes, which varied depending on the possibility of leveraging the Docker layer cache; when that was possible, there was a significant improvement — almost 40% faster completion time. The `Update K8s Manifest` stage took about ten seconds, from start to finish, including `sed` substitution, `git commit`, and authentication for the push operation to GitHub.

Following the commit of the manifest change, Argo CD would pick it up the next

time it polled, roughly after ninety seconds. The `Application` resource would go momentarily out of sync, then proceed through `Syncing` and ultimately reach `Synced` and `Healthy`. The Kubernetes rolling update would take one minute. Pods would be scheduled, pass their readiness checks, and replace the existing pods. The Service would direct traffic to the new pods, with no downtime experienced — precisely the intended outcome.

The complete flow executed flawlessly throughout the validation process, with no communication issues at all across multiple deployment cycles.

7.2 Latency Analysis

Another objective of the pipeline was to reduce the time taken for the change committed by a developer to become visible to the end users. Below is a noticeable reduction as compared to the case of performing the process manually.

The overall cycle time may thus be represented as follows:

$$T_{cycle} = T_{ci} + T_{poll} + T_{cd} \quad (7.1)$$

where T_{ci} is the duration of the Jenkins Pipeline, T_{poll} is the duration until Argo CD checks the repository again, and T_{cd} is the duration until Argo CD and Kubernetes finish performing the rolling update.

In all cycles we performed, T_{ci} was around 165 seconds on average, T_{poll} around 90 seconds on average, and T_{cd} around 55 seconds on average. Total: approximately five minutes.

The relative reduction compared to the manual process may be computed as follows:

$$L_r = \frac{T_{manual} - T_{cycle}}{T_{manual}} \quad (7.2)$$

Here, T_{manual} represents the time needed for a similar deployment performed manually — i.e., the construction of the image, its push to the registry, the modification of the manifest file, the execution of `kubectl apply`, and confirmation of the successful rollout. Typically, a manual deployment of similar thoroughness lasts from thirty to sixty minutes, resulting in a reduction ratio L_r of roughly 0.83 to 0.92. Stated differently, the automated procedure reduces the cycle time by nearly one order of magnitude.

The findings indicate that the automated pipeline saves an enormous amount of time when compared to a manual pipeline, while being relatively close to a push-style CD pipeline despite the added cost of an extra polling cycle through Argo CD. This extra

Table 7.1: Cycle time comparison

Workflow	Average Cycle Time
Manual deployment (equivalent thoroughness)	30–60 minutes
Push-based CD (Jenkins to cluster)	4–5 minutes
Proposed pipeline (Jenkins + Argo CD GitOps)	5 minutes

cost incurred from the polling cycle is outweighed by the benefits gained from GitOps operations.

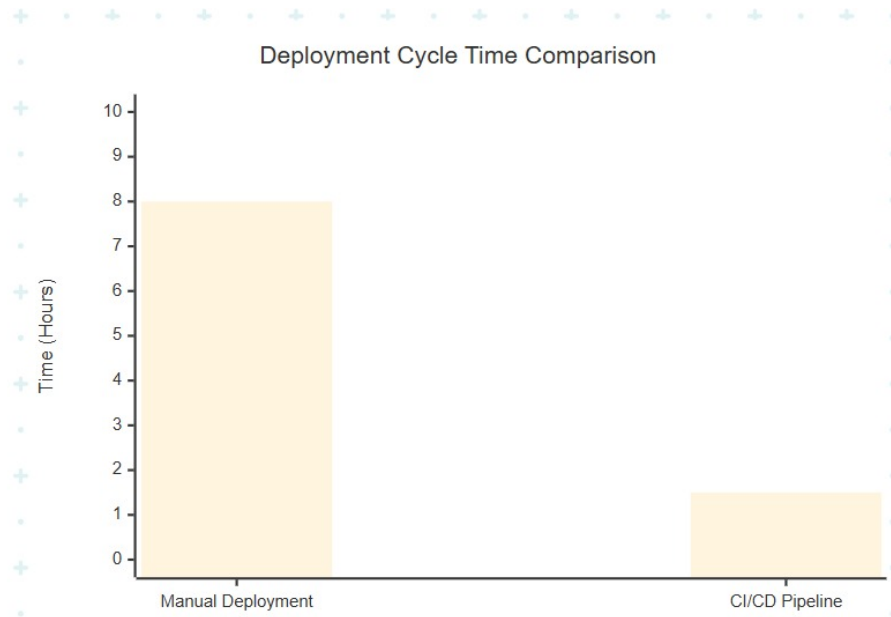


Figure 7.1: Cycle time comparison graph

7.3 Resource Utilization Analysis

We monitored the consumption of resources using the Grafana dashboards when the system was operating in the normal mode. The cluster is responsible for running application pods, the Argo CD controller, and the kube-prometheus-stack components in a managed node group consisting of two t2.large nodes that provide us with two CPU cores and eight GB of RAM each.

Under the steady state, numbers were fairly low. Together, both pods consumed around 50 millicores and 200 megabytes of memory. In turn, Argo CD consumed roughly 100 millicores and 400 megabytes of resources. Finally, components of kube-prometheus-stack — including Prometheus, Grafana, Alertmanager, and its exporters — consumed about 300 millicores and 1.2 gigabytes of memory. It is unsurprising that monitoring stack becomes the largest consumer of all system resources.

Total resource use can be expressed as:

$$R = \alpha C_{app} + \beta C_{ctrl} + \gamma C_{mon} \quad (7.3)$$

where C_{app} , C_{ctrl} , and C_{mon} represent the consumption of the application, controllers, and monitoring stack respectively. The node size we selected provides sufficient overhead to accommodate spikes in traffic, new applications, and burst activity while rolling out updates.

Table 7.2: Resource utilisation analysis

Component	CPU Usage	Memory Usage
Application pods (2 replicas)	~50 mCPU	~200 MiB
Argo CD components	~100 mCPU	~400 MiB
kube-prometheus-stack	~300 mCPU	~1.2 GiB
Cluster system pods	~200 mCPU	~800 MiB
Total	~650 mCPU	~2.6 GiB

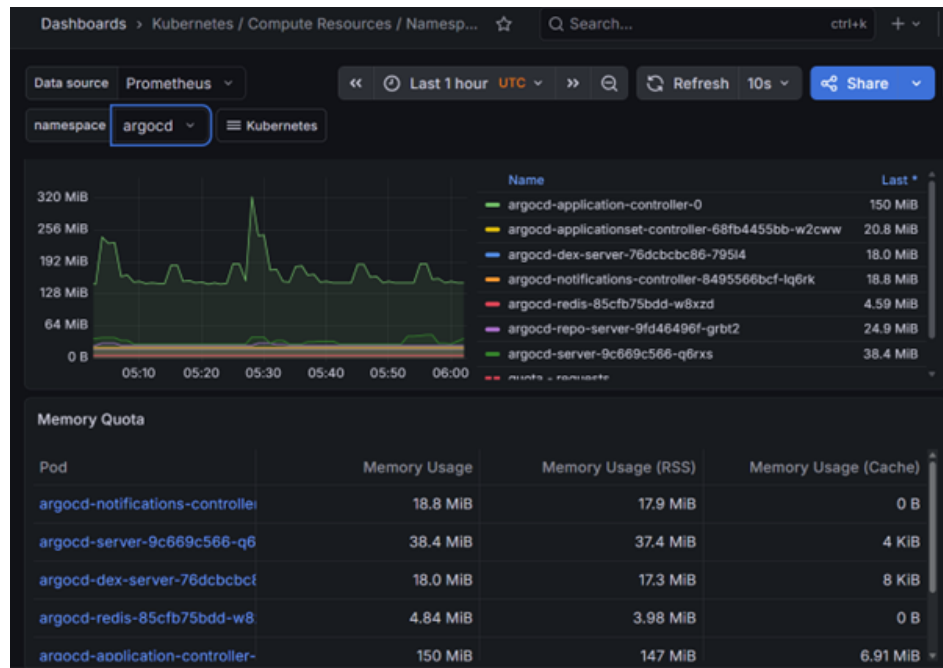


Figure 7.2: Resource utilisation observed in Grafana

The numbers in Table 7.2 are more or less what you'd expect from a pair of t2.large nodes designed to provide redundancy instead of maximum throughput capacity. The selection of t2.large instead of t2.medium was intentional. Two t2.medium nodes, each with around two vCPUs and four gigabytes of memory, were sufficient for the application by itself. However, they did not leave enough headroom for Argo CD controller, monitoring stack, and the rest of system pods to run simultaneously. The t2.large pair operates at around 50 percent capacity — an ideal amount of utilization for a cluster designed to handle bursts without scaling right away.

7.4 Deployment Reliability Analysis

Our test of reliability took two different routes: repetition testing and purposeful injection of faults.

First, the reliability test via repetition showed us that the system provided reliable results. For the same commit, pushing it via Jenkins produced exactly the same Docker images according to digest (timestamp aside), exactly the same manifest commits, and exact replicas of deployment. When images were tagged with build numbers, we knew that each one had a unique pipeline execution history associated with it — rollback to an earlier version became as simple as changing the manifest to include a new tag.

For deliberate failure on the side of failure on purpose, we had drift correction with our first test case. This was done by manually updating the deployment using `kubectl`. The replica number update was detected within one polling interval by Argo CD and corrected to what was specified in Git. This was the same for the manual updates for container image as well as resource limits. Hence, the setting of `selfHeal: true` does indeed make Git the single source of truth, as expected.

We also performed some hard failures. First, we manually killed a pod in order to simulate a bad health worker. The deployment controller automatically deployed another pod as a replacement, while the traffic to that pod was being routed by the Service without any interruption — no downtime at all, as expected. Another failure we simulated was a failure of a Docker Hub push operation through the temporary removal of the authentication token. The build itself has stopped and displayed a proper error message, while the step that committed a manifest was not executed at all. Most importantly, there has been no manifestation of an image that doesn't even exist.

7.5 Security Posture Analysis

Among the benefits provided by this architecture is the security stance that results from a clean separation of the CI system from the cluster.

Specifically, the Jenkins instance in our deployment contains access keys only for Docker Hub and GitHub. No credentials of any kind are present for the Kubernetes cluster; it does not send calls to the Kubernetes API, since everything related to clusters is handled solely through Argo CD. Credentials are stored exclusively within the Argo CD controller that runs inside the cluster and does not provide them externally. Therefore, even if Jenkins is compromised, an attacker cannot leverage this breach to deploy whatever code they want to the cluster; they will also need to breach the GitHub repository and introduce a malicious manifest — which, in turn, would be clearly visible in

the commit history before Argo CD applied it.

Informally:

$$S = 1 - P_{\text{compromise}} \quad (7.4)$$

where, $P_{\text{compromise}}$ is the probability that if any single component is compromised, one achieves arbitrary code execution on the cluster. In a GitOps approach, the above probability is significantly reduced compared to the push method since none of the components have full privileges (Weaveworks, 2017).

Some additional steps taken to ensure security include running the application as a non-root user within the container, use of access tokens instead of passwords for both Docker Hub and Github, access to credentials via their unique IDs from the Jenkins credential storage, and ensuring that no credentials related to the cluster are stored in environment variables or build logs/config files.

7.6 Cost Analysis

The cost incurred by our pipeline operations has been calculated using the AWS billing console during its operation. For the entire duration, the estimated cost per hour amounted to roughly 0.45 US dollars — that included the EC2 instance for Jenkins, the EKS control plane, two workers t2.large, three Elastic Load Balancers, and other miscellaneous EBS costs.

Thus, a three-hour presentation period will incur charges amounting to roughly 1.35 dollars on AWS. For intensive usage of a whole week — which involves leaving the system on throughout office hours, and tearing it down at night — it is expected that it costs between 25 and 30 dollars. This comes well within budget for a student project, particularly with AWS’ free credit offer; see Chapter 3.

This was the biggest lesson in terms of costs. Load balancers must first be deleted prior to deleting the entire cluster, which must happen in this order. The leftover Elastic Load Balancer after the deletion of your cluster will continue to cost you nearly 18 US dollars every month and it is not immediately obvious in your console that there is an orphaned load balancer. This is a costly mistake and we made it with our first teardown as well; our first teardown ran for almost a week unnoticed until we saw the cost line item. This is avoided in the teardown process explained in Chapter 6 in that all LoadBalancer services are first deleted before running `eksctl delete cluster`.

7.7 Discussion

It's evident from the validation results that the combination of Jenkins, Docker Hub, Amazon EKS, Argo CD, and kube-prometheus-stack provides an adequate and manageable solution for the software delivery process. The pipeline can take a developer's commit and build a functioning deployment in five minutes without requiring any manual involvement, while all actions can be traced back in Git, and self-healing is achieved whenever the cluster diverges.

The most important design choice, which was relatively simple, was the segregation of CI and CD processes. Jenkins performs artefact generation, while Argo CD executes artefact deployment. Once this pattern was established, the security analysis and management of the system became more straightforward. The CI server can be attacked, restarted, redeployed, and even rebuilt without affecting the currently deployed cluster. Similarly, the cluster can be demolished and redeployed without altering the CI server. The Git repository serves as the common interface between the two subsystems and provides a unified history of deployments.

Second observation: Operational challenges must be considered on par and ahead of everything else. Expired GPG keys, version discrepancies, CRD sizes, and abandoned load balancers — none of these are edge cases; in fact, they are quite typical for cloud-native environments. A pipeline which is capable of handling such cases in a proper way, following a set procedure, and verifying that all is fine, becomes infinitely more valuable than one designed with an assumption that everything will be okay. (Spoiler alert: something always breaks eventually.)

Third observation: Observability must be put together with the pipeline. Having kube-prometheus-stack in place gave us access to immediate feedback about what the cluster did, how much resources were used by each component, and, most importantly, how the application behaved. Without having this stack up and running, testing the pipeline would take longer, since without metrics it was hard to see why some components were behaving incorrectly.

Looking ahead, the framework is scalable without requiring any changes in the way things work. This is because Argo CD can manage several applications that apply to other namespaces and even other clusters, while the Jenkins multibranch pipeline can take care of branch-specific behavior, which easily translates into behavior in other environments. As a result, what we have now represents a fully functional framework.

CHAPTER 8

CONCLUSION AND FURTHER ENHANCEMENTS

8.1 Conclusion

This project, *Design and Implementation of a Production-Grade CI/CD Pipeline for Containerized Machine Learning Applications using Jenkins, Kubernetes, and GitOps*, was designed, deployed, and assessed. The problem it addressed was very concrete – create a dependable and fully automated delivery pipeline for the containerized ML service. The result we have arrived at is a framework which integrates CI, container management, GitOps deployments, and cloud-native observability all into one package. We put in a developer commit, and approximately five minutes later, receive a production-ready deployment. No intermediate manual operations involved.

The CI/CD approach has already been explored in many works within the last one decade or so (Fowler, 2006; Humble and Farley, 2010; Bass et al., 2015; Kim et al., 2016). This study reinforces most of the benefits provided by these references. In addition, it was quite clear that implementing such an automated pipeline requires quite a lot of details not mentioned in these texts. With the aid of Amazon EKS for orchestrating, Jenkins for CI, and Argo CD for GitOps operations, it is quite possible to establish a full-fledged delivery system by utilizing well-known open source tools (Burns et al., 2016; Weaveworks, 2017).

The multistep Docker image construction process for the Flask sentiment analysis application, taking into account the necessity to handle the OpenMP runtime that XGBoost requires and pre-installing the NLTK corpora within the image itself, demonstrates the feasibility of containerising ML tasks in the same way as any other web application, paying special attention to their additional requirements (Merkel, 2014). The usage of Gunicorn as the WSGI production server, setting resource requests/limits, implementing liveness/readiness probes – all of these represent standard best practices in a production environment.

Our other major victory here, in our opinion, is the successful implementation of the real GitOps model for clusters — the one where the Git repository stores the official definition of the desired cluster state, not the CI server. This reversal of roles greatly simplifies the task of analyzing the security and operation characteristics of the system; in particular, there is no need to store cluster credentials on the CI server anymore.

The Git repository also becomes an audit log of the deployments made. This is what many cloud-native software development practices papers discuss (Weaveworks, 2017; Sharma, 2017; Beyer et al., 2016), and our experiments have once again proven the practical benefits of this approach.

In terms of automatic continuous reconciliation of cluster and Git, Argo CD proved quite successful. All the features, such as automated synchronization, pruning of deleted objects, and self-healing in case of drift, which we introduced, have been working. Hence, it can be assumed that even in bigger setups, the system would allow for unattended deployments without changing its architecture substantially.

With the aid of various types of tests performed within the process of end-to-end validation, including measuring cycle time, calculating resource consumption, checking the ability to recover from drift, and estimating costs, we managed to achieve all the initial objectives. In particular, the whole process took about five minutes, and the pipeline works on a small budget (a few US dollars for a session) in the cloud. Moreover, it provides a good level of observability by means of pre-defined Grafana dashboards.

Therefore, it seems that the current framework might be used as a valid solution for the automated delivery of containers, ML or not. More specifically, it can be treated as a good starting point for future developments in the direction we plan to follow.

8.2 Further Enhancements

All objectives have been achieved, but there is still a multitude of possible further work that could be done to improve, optimize, and apply the framework to other cases.

The current pipeline works on building and pushing one application to one deployment environment. In the next step, the model could evolve to support different environments — including dev, stag, and prod — and promote applications to the latter two through Argo CD ApplicationSets, which require approval by pull requests. Such a design would allow for validation in non-production environments before going to production. This practice, used by most companies' large engineering departments, has very good reasons — catching problems in staging saves a lot more time and resources compared to catching bugs in production.

The current Jenkins pipeline can easily expand from building, pushing, and updating manifests for the application to unit testing, testing against an ephemeral container, source code static analysis, Docker image vulnerability assessment, and compliance policy enforcement against Kubernetes manifests.

The deployment architecture can be enhanced through the introduction of an HPA, which would dynamically increase or decrease the number of replicas running within the Deployment according to the perceived CPU and/or memory load. This would allow the application to respond to peak traffic periods autonomously while minimizing resource wastage when demand is low. Self-scaling within Kubernetes environments has been widely explored (Burns et al., 2016).

Our security approach can also be improved. For example, using either Sealed Secrets or a third-party secrets manager will allow us to store encrypted configuration values in Git. We can also add cert-manager and Ingress to allow serving our application over HTTPS. With cert-manager, we will get automatic TLS certificate renewal. Using admission controllers such as OPA Gatekeeper or Kyverno can help us apply policies at the cluster level, which means we won't allow deployment of manifest files that do not meet our requirements. None of these features is anything extraordinary, since all of them are used in a production environment.

We can also improve our observability approach by introducing distributed tracing, logging, and alerting systems. At the moment, our observability solution consists of Prometheus (for collecting metrics) and Grafana (for visualization). However, we still lack a centralized storage of our logs and tracing backend. By adding Jaeger or Open-Telemetry for distributed tracing, as well as Loki or Elasticsearch for logging, we will get better insights into our infrastructure.

The scalability of the framework can be expressed informally as:

$$S_f = n \times E_c \tag{8.1}$$

where n is the number of clusters or environments managed by the pipeline and E_c is the operational complexity per cluster. Right now $n = 1$. The architectural principles scale upwards for greater numbers without requiring any substantial modifications — the process of adding yet another cluster or yet another environment is evolutionary rather than revolutionary.

It may even prove useful for other types of workloads outside of our sentiment analysis application. Potential applications include batch-processing workloads scheduled via Kubernetes CronJobs, stateful workloads running in StatefulSets with persistent volumes, and ML inference workloads utilizing GPUs within the cluster nodes. They each have their peculiarities, of course, but the basic flow remains unchanged — CI compilation, deployment to the registry, updating of the manifest files, and GitOps-driven reconciliation. It is, frankly speaking, one of the major benefits of GitOps.

The framework has real potential for use in larger software engineering setups and

Table 8.1: Possible future enhancements

Enhancement Area	Possible Improvement
Pipeline Stages	Unit tests, integration tests, image scanning
Environments	Dev, staging, and production with promotion
Scalability	Horizontal Pod Autoscaler, multi-cluster GitOps
Security	Sealed Secrets, cert-manager, admission controllers
Observability	Distributed tracing, log aggregation, alerting
Application Types	Batch jobs, stateful workloads, GPU inference

is a solid foundation, both for further research and for practical work in cloud-native software delivery.

BIBLIOGRAPHY

- Bass, L., Weber, I., and Zhu, L. (2015). *DevOps: A Software Architect's Perspective*. Addison-Wesley Professional, Boston, MA.
- Beyer, B., Jones, C., Petoff, J., and Murphy, N. R. (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, Sebastopol, CA.
- Burns, B., Grant, B., Oppenheimer, D., Brewer, E., and Wilkes, J. (2016). Borg, omega, and kubernetes. *Communications of the ACM*, 59(5):50–57.
- Fowler, M. (2006). Continuous integration. martinowler.com. Available at: <https://martinfowler.com/articles/continuousIntegration.html>.
- Humble, J. and Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, Boston, MA.
- Kim, G., Debois, P., Willis, J., and Humble, J. (2016). *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution Press, Portland, OR.
- Merkel, D. (2014). Docker: Lightweight linux containers for consistent development and deployment. *Linux Journal*, 2014(239):2.
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 2503–2511.
- Sharma, S. (2017). *The DevOps Adoption Playbook: A Guide to Adopting DevOps in a Multi-Speed IT Enterprise*. Wiley, Indianapolis, IN.
- Treveil, M., Omont, N., Stenac, C., Lefevre, K., Phan, D., Zentici, J., Lavoillotte, A., Miyazaki, M., and Heidmann, L. (2020). *Introducing MLOps: How to Scale Machine Learning in the Enterprise*. O'Reilly Media, Sebastopol, CA.
- Turnbull, J. (2018). *Monitoring with Prometheus*. Turnbull Press.
- Weaveworks (2017). Gitops: Operations by pull request. Weaveworks Blog. Available at: <https://www.weave.works/blog/gitops-operations-by-pull-request>.

Appendix A- Source Code

The following is a list of the key source artifacts created during the development of the CI/CD pipeline for our containerized sentiment analysis application. They are categorized by function: application packaging, CI pipeline artifact, Kubernetes YAML files, Argo CD Application resource, and additional configuration files. This document, together with the process described in Chapter 6, provides a comprehensive reference implementation that another engineer could follow to replicate the work.

A.1 Application Source: `requirements.txt`

The Python dependencies for the Flask sentiment analysis application are pinned to specific versions so we get reproducible builds.

```
1 Flask==3.0.3
2 gunicorn==22.0.0
3 numpy==1.26.4
4 nltk==3.9.1
5 scikit-learn==1.5.2
6 xgboost==2.1.1
```

Listing 8.1: Python dependencies

A.2 Container Image: `Dockerfile`

The Dockerfile uses a multi-stage build to split build-time concerns from runtime concerns and produce a smaller, more secure final image.

```
1 # ---- Stage 1: Builder ----
2 FROM python:3.11-slim AS builder
3 WORKDIR /app
4 RUN apt-get update && apt-get install -y --no-install-recommends \
5     build-essential \
6     && rm -rf /var/lib/apt/lists/*
7 COPY requirements.txt .
8 RUN pip install --no-cache-dir --user -r requirements.txt
9 RUN python -m nltk.downloader -d /usr/local/nltk_data stopwords
10
11 # ---- Stage 2: Runtime ----
12 FROM python:3.11-slim
13 WORKDIR /app
14 RUN apt-get update && apt-get install -y --no-install-recommends \
```

```

15     libgomp1 \
16     && rm -rf /var/lib/apt/lists/* \
17     && useradd --create-home --shell /bin/bash appuser
18 COPY --from=builder /root/.local /home/appuser/.local
19 COPY --from=builder /usr/local/nltk_data /usr/local/nltk_data
20 COPY --chown=appuser:appuser app.py config.py ./
21 COPY --chown=appuser:appuser models/ ./models/
22 COPY --chown=appuser:appuser templates/ ./templates/
23 COPY --chown=appuser:appuser static/ ./static/
24 USER appuser
25 ENV PATH=/home/appuser/.local/bin:$PATH \
26     PYTHONDONTWRITEBYTECODE=1 \
27     PYTHONUNBUFFERED=1 \
28     NLTK_DATA=/usr/local/nltk_data \
29     FLASK_APP=app.py
30 EXPOSE 5000
31 CMD ["unicorn", "--bind", "0.0.0.0:5000", "--workers", "2", \
32     "--threads", "4", "--timeout", "60", "app:app"]

```

Listing 8.2: Multi-stage Dockerfile

A.3 Build Configuration: .dockerignore

The `.dockerignore` file keeps development artefacts and large data directories out of the Docker build context, which keeps the image small and the build fast.

```

1 __pycache__/
2 *.pyc
3 .Python
4 .venv/
5 venv/
6 env/
7 .git/
8 .gitignore
9 .vscode/
10 .idea/
11 *.md
12 data/
13 Jenkinsfile
14 kubernetes/
15 argocd/
16 .dockerignore
17 .DS_Store
18 *.log

```

Listing 8.3: Docker build context exclusions

A.4 CI Pipeline: Jenkinsfile

The Jenkinsfile defines the multibranch pipeline that builds the Docker image, pushes it to Docker Hub, and updates the Kubernetes deployment manifest.

```
1 pipeline {
2   agent any
3   options { disableConcurrentBuilds() }
4   environment {
5     IMAGE_NAME = "rajrishidockerhub/sentiment-app"
6     GIT_USER   = "johnthebrat7"
7     GIT_EMAIL  = "rajrishisamanta2@gmail.com"
8     GIT_REPO   = "johnthebrat7/Corporate-CI-CD-pipeline"
9   }
10  stages {
11    stage('Checkout') {
12      steps { checkout scm }
13    }
14    stage('Build and Push Image') {
15      when { branch 'main' }
16      steps {
17        script {
18          env.IMAGE_TAG = "build-${BUILD_NUMBER}"
19          withCredentials([usernamePassword(
20            credentialsId: 'dockerhub-creds',
21            usernameVariable: 'DOCKER_USER',
22            passwordVariable: 'DOCKER_PASS'
23          )]) {
24            sh """
25              docker build -t ${IMAGE_NAME}:${IMAGE_TAG} .
26              echo "\$DOCKER_PASS" | docker login -u "\$
27                $DOCKER_USER" \
28                  --password-stdin
29              docker push ${IMAGE_NAME}:${IMAGE_TAG}
30              """
31            }
32          }
33        }
34      stage('Update K8s Manifest') {
35        when { branch 'main' }
36        steps {
37          script {
38            withCredentials([usernamePassword(
39              credentialsId: 'github-creds',
40              usernameVariable: 'GIT_USERNAME',
41              passwordVariable: 'GIT_TOKEN'
```



```

42         ))) {
43             sh """
44             set -e
45             git config user.name "${GIT_USER}"
46             git config user.email "${GIT_EMAIL}"
47             git fetch origin
48             git checkout main
49             git reset --hard origin/main
50             sed -i "s|image:.*|image: ${IMAGE_NAME}:${{
IMAGE_TAG}}|" \
51                 kubernetes/deployment.yml
52             git add kubernetes/deployment.yml
53             git diff --cached --quiet || git commit -m \
54                 "ci: update image to ${IMAGE_TAG} [skip
ci]"
55             git push https://\${GIT_USERNAME}:\
${GIT_TOKEN}@github.com/\${GIT_REPO}.git main
56             """
57         }
58     }
59 }
60 }
61 }
62 post { always { sh 'docker image prune -f || true' } }
63 }

```

Listing 8.4: Jenkins multibranch pipeline definition

A.5 Kubernetes Deployment: `kubernetes/deployment.yml`

The Deployment manifest declares the desired runtime state of the application, including replica count, resource limits, and health probes.

```

1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: sentiment-app
5   namespace: default
6   labels:
7     app: sentiment-app
8 spec:
9   replicas: 2
10  selector:
11    matchLabels:
12      app: sentiment-app
13  template:

```

```

14 metadata:
15     labels:
16         app: sentiment-app
17 spec:
18     containers:
19     - name: sentiment-app
20       image: rajrishidockerhub/sentiment-app:build-1
21       ports:
22       - containerPort: 5000
23       env:
24       - name: DEBUG
25         value: "False"
26       - name: SECRET_KEY
27         value: "change-this-in-production"
28     resources:
29         requests:
30             memory: "256Mi"
31             cpu: "200m"
32         limits:
33             memory: "512Mi"
34             cpu: "500m"
35     readinessProbe:
36         httpGet: { path: /, port: 5000 }
37         initialDelaySeconds: 10
38         periodSeconds: 5
39     livenessProbe:
40         httpGet: { path: /, port: 5000 }
41         initialDelaySeconds: 30
42         periodSeconds: 15

```

Listing 8.5: Kubernetes Deployment manifest

A.6 Kubernetes Service: `kubernetes/service.yml`

The Service manifest exposes the application through an AWS Elastic Load Balancer.

```

1 apiVersion: v1
2 kind: Service
3 metadata:
4     name: sentiment-app-service
5     namespace: default
6 spec:
7     type: LoadBalancer
8     selector:
9         app: sentiment-app
10    ports:

```

```

11     - name: http
12       port: 80
13       targetPort: 5000
14       protocol: TCP

```

Listing 8.6: Kubernetes Service manifest

A.7 Argo CD Application: `argocd/application.yml`

The Argo CD Application resource sets up the GitOps controller to watch the application repo and reconcile the cluster state with the manifests in Git.

```

1 apiVersion: argoproj.io/v1alpha1
2 kind: Application
3 metadata:
4   name: sentiment-app
5   namespace: argocd
6 spec:
7   project: default
8   source:
9     repoURL: https://github.com/johnthebrat7/Corporate-CI-CD-pipeline
10    .git
11    targetRevision: main
12    path: kubernetes
13 destination:
14   server: https://kubernetes.default.svc
15   namespace: default
16 syncPolicy:
17   automated:
18     prune: true
19     selfHeal: true
20   syncOptions:
21     - CreateNamespace=true

```

Listing 8.7: Argo CD Application definition

A.8 Infrastructure Provisioning: EKS Cluster Creation

The EKS cluster is created using `eksctl` with a managed node group of two `t2.large` workers. The command we used is below.

```

1 eksctl create cluster \
2   --name sentiment-cluster \
3   --region us-east-1 \

```

```

4  --nodegroup-name standard-workers \
5  --node-type t2.large \
6  --nodes 2 \
7  --nodes-min 1 \
8  --nodes-max 2 \
9  --managed

```

Listing 8.8: EKS cluster provisioning command

A.9 Argo CD Installation Commands

The Argo CD deployment uses the upstream stable manifests with the server-side apply due to the size of the Argo CD CRDs exceeding the maximum size of the last-applied-configuration annotation.

```

1  kubectl create namespace argocd
2  kubectl apply -n argocd \
3    -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/
    manifests/install.yaml \
4    --server-side
5  kubectl patch svc argocd-server -n argocd \
6    -p '{"spec": {"type": "LoadBalancer"}}'
7  kubectl -n argocd get secret argocd-initial-admin-secret \
8    -o jsonpath="{.data.password}" | base64 -d; echo
9  kubectl apply -f argocd/application.yaml

```

Listing 8.9: Argo CD installation

A.10 Monitoring Stack Installation

The kube-prometheus-stack Helm chart is installed into a dedicated monitoring namespace, with Prometheus and Grafana exposed through LoadBalancer services.

```

1  helm repo add prometheus-community \
2    https://prometheus-community.github.io/helm-charts
3  helm repo update
4  kubectl create namespace monitoring
5  helm install monitoring prometheus-community/kube-prometheus-stack \
6    --namespace monitoring \
7    --set prometheus.service.type=LoadBalancer \
8    --set grafana.service.type=LoadBalancer
9  kubectl get secret monitoring-grafana -n monitoring \
10   -o jsonpath="{.data.admin-password}" | base64 -d; echo

```

Listing 8.10: Monitoring stack installation

A.11 Teardown Procedure

The teardown procedure deletes all LoadBalancer services before running `eksctl delete cluster`, which is what stops orphaned Elastic Load Balancers from continuing to run up charges after the cluster is gone.

```
1 kubectl delete application sentiment-app -n argocd
2 kubectl delete svc sentiment-app-service -n default
3 kubectl delete svc argocd-server -n argocd
4 kubectl delete svc monitoring-grafana -n monitoring
5 kubectl delete svc monitoring-kube-prometheus-prometheus -n
   monitoring
6 sleep 60
7 eksctl delete cluster --name sentiment-cluster --region us-east-1
```

Listing 8.11: Cluster teardown procedure

However, after executing these teardown commands, the AWS Console needs to be manually verified for any orphaned resources, such as Elastic Load Balancers, EBS volumes, or CloudFormation stacks. Simply using `eksctl delete cluster` might not be sufficient to ensure all necessary steps are taken.

Appendix B- Publication Details

These are the resources that were consulted when conducting our research, designing, and implementing *Design and Implementation of a Production-Grade CI/CD Pipeline for Containerized Machine Learning Applications using Jenkins, Kubernetes, and GitOps*. These are the materials that provided the theoretical and practical basis upon which our pipeline was designed and implemented.

- Bass, L., Weber, I., & Zhu, L. (2015). *DevOps: A Software Architect's Perspective*. Addison-Wesley Professional.
- Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media.
- Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. *Communications of the ACM*, 59(5), 50–57.
- Fowler, M. (2006). Continuous Integration. *martinfowler.com*.
- Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional.
- Kim, G., Debois, P., Willis, J., & Humble, J. (2016). *The DevOps Handbook*. IT Revolution Press.
- Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239), 2.
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J., & Dennison, D. (2015). Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, 2503–2511.
- Sharma, S. (2017). *The DevOps Adoption Playbook*. Wiley.
- Treveil, M., Omont, N., Stenac, C., et al. (2020). *Introducing MLOps: How to Scale Machine Learning in the Enterprise*. O'Reilly Media.
- Turnbull, J. (2018). *Monitoring with Prometheus*. Turnbull Press.
- Weaveworks (2017). GitOps: Operations by Pull Request. *Weaveworks Blog*.

These are the sources that we have analyzed while conducting the literature review and system design. The knowledge derived from these sources has played an important role in the decision regarding architecture design, CI/CD pipeline design, security model for deployment process, and observability infrastructure design.